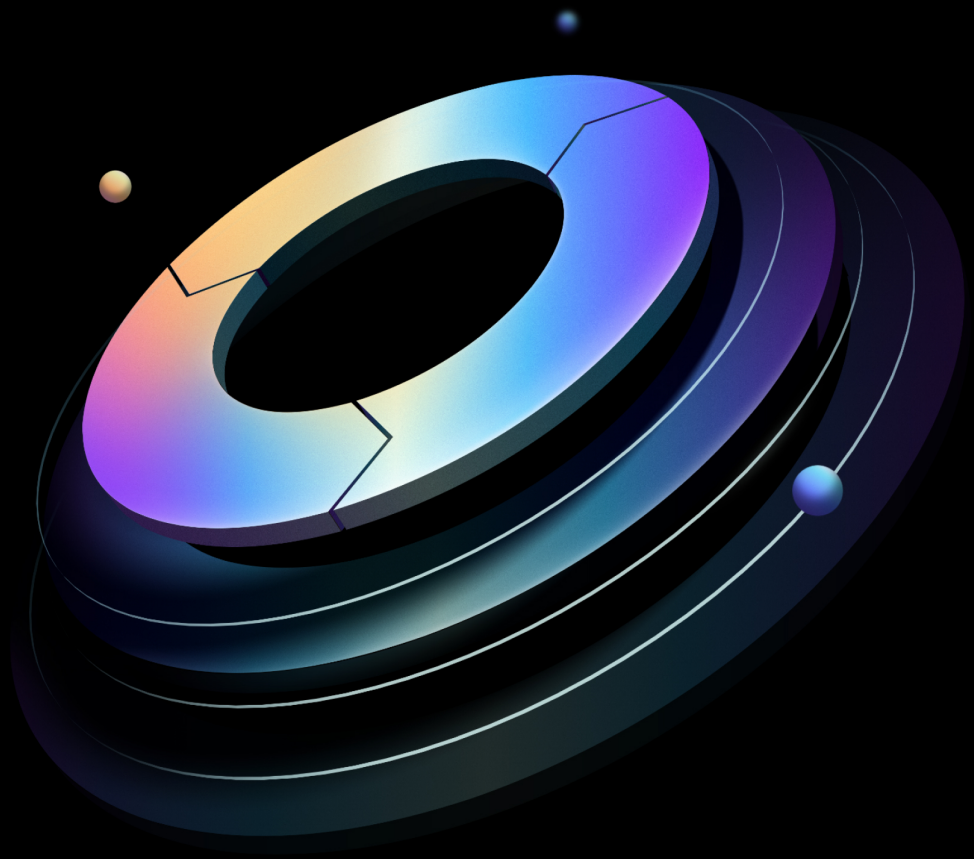
HashiCorp

Vault Installation Guide - Virtual machines

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **5**
1.1 Why use HashiCorp Validated Designs? . . . 5
1.2 Audience . . . 5
1.3 Supported platforms and versions . . . 6
1.4 Definitions and concepts . . . 6
1.5 Organizational requirements . . . 9
1.6 Guide relationship . . . 10

2 Architecture **10**
2.1 Cluster . . . 11
2.1.1 Integrated storage . . . 11
2.1.2 Clustering . . . 11
2.2 Load balancing . . . 13
2.2.1 TLS . . . 14
2.3 Backup snapshot storage . . . 14
2.4 Sealing and unsealing . . . 15
2.4.1 Auto-Unseal using cloud-based KMS services . . . 15
2.4.2 Auto-unseal using a hardware security module . . . 16
2.5 Audit devices . . . 16
2.6 Cluster architecture summary . . . 17
2.7 Disaster recovery secondary cluster . . . 19
2.7.1 Failover mechanism . . . 19

3 Detailed design **21**
3.1 Core design requirements . . . 21
3.1.1 Compute . . . 22
3.1.2 Networking . . . 23
3.1.3 Software . . . 29
3.1.4 Storage . . . 30
3.1.5 Configuration . . . 31
3.1.6 Unseal and recovery keys . . . 36
3.2 Compute . . . 39
3.2.1 Operating systems . . . 39

3.2.2 Sizing . . . 40
3.2.3 Autoscaling . . . 41
3.3 Networking . . . 41
3.3.1 Private network . . . 41
3.3.2 Subnets . . . 42
3.3.3 Routing . . . 42
3.3.4 Network security groups . . . 42
3.3.5 Load balancing . . . 42
3.4 Software . . . 43
3.4.1 Vault license . . . 43
3.5 Storage . . . 43
3.6 Auto-unseal . . . 43
3.7 Domain name system . . . 43
3.8 Certificates . . . 44
3.9 IAM . . . 44

4 Deploying Vault using Terraform 46
4.1 Platform-specific guidance . . . 46
4.2 Deployment sequence overview . . . 48
4.3 Preparation . . . 49
4.3.1 Create the certificate files . . . 49
4.3.2 Obtain the Vault Enterprise license file . . . 49
4.3.3 Download and Install the Vault CLI . . . 49
4.3.4 Download and install the Terraform CLI . . . 50
4.3.5 Download the Terraform module . . . 50
4.3.6 Configure cloud credentials . . . 50
4.4 Installation . . . 51
4.4.1 Initialize Terraform . . . 51
4.4.2 Configure variables for deployment . . . 52
4.4.3 Create and apply Terraform plan . . . 52
4.4.4 Validate installation . . . 52
4.5 Initialize . . . 53
4.5.1 Create GPG public keys . . . 53
4.5.2 Initialize Vault . . . 53

5 Deploying Vault Enterprise in a private datacenter 54
5.1 Overview . . . 54
5.2 Prepare . . . 55
5.2.1 TLS . . . 55
5.2.2 License . . . 56
5.2.3 Server configs . . . 56
5.3 Build . . . 60
5.3.1 Servers . . . 60
5.3.2 Load balancer . . . 60
5.3.3 Snapshot storage . . . 60
5.4 Install Vault . . . 60
5.4.1 Initialize . . . 63
5.4.2 Initialize Vault . . . 63
5.4.3 Form the cluster . . . 65

6 Post-installation configuration 66
6.1 Overview . . . 66
6.1.1 Prerequisites . . . 66
6.2 Configure audit logs . . . 67
6.2.1 Types of audit devices . . . 67
6.2.2 Multiple audit devices . . . 69
6.2.3 Enabling an audit device . . . 70
6.3 Namespaces . . . 73
6.3.1 What are Vault namespaces . . . 74
6.3.2 When to use Vault namespaces . . . 74
6.3.3 Useful resources . . . 75

7 Disaster recovery initial setup 75
7.1 Overview . . . 76
7.1.1 Prerequisites . . . 76
7.2 Types of cluster replication in Vault Enterprise . . . 76
7.3 Setting up DR replication . . . 77
7.3.1 Configure DR operation token . . . 78

8 Performance replication architecture and setup 79
8.1 Overview . . . 79

8.2 Architecture . . . 79
8.3 Setting up performance replication . . . 84
8.3.1 Prerequisites . . . 84
8.3.2 Set up process . . . 85

9 Observability setup 86
9.1 Telemetry endpoint setup . . . 86
9.2 Health endpoint setup . . . 86
9.3 Handoff to administration . . . 86

10 Next steps 87

11 Changelog 87
11.1 2026-08 . . . 87
11.2 2026-03 . . . 87
11.3 2026-01 . . . 87
11.4 2025-12 . . . 87
11.5 2025-10 . . . 88
11.6 2025-05 . . . 88
11.7 2025-04 . . . 88
11.8 2025-01 . . . 88
11.9 2024-02 . . . 88

12 Credits 88

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

1.2 Audience

This document is intended primarily for practitioners (including platform, networking, identity, and Infosec teams) who want to deploy Vault Enterprise clusters on-premises or on cloud infrastructure.

After using the Installation Guide to successfully deploy your Vault Enterprise infrastructure, Vault operators should refer to Vault: validated design guidance. Using the Administration Guide, Vault operators can learn how to integrate application platforms, configure machine and user access, centralize secrets, perform upgrades, and more.

1.3 Supported platforms and versions

As of HVD Document Version 1.0, the Installation Guide supports deploying Vault Enterprise on the following compute platforms:

- AWS on EC2 instances
- Azure on VM instances
- GCP on GCE
- Bare-metal or VM-based servers in a private datacenter

This guide applies to Vault Enterprise versions `vault_1.17.3+ent` and later, and can be downloaded from HashiCorp Releases or the HashiCorp Developer page.

While Vault Enterprise is officially supported for a number of different operating systems and processor architectures, this guide primarily uses `linux/amd64` within its examples, unless otherwise specified. Please note that HSM support is currently only available for `linux/amd64` platforms.

1.4 Definitions and concepts

This documentation intentionally uses technology agnostic terminology. However, there are some terms which do not translate perfectly between providers. The following are the definitions of terms this document uses:

Term	Definition
Availability zone	An availability zone (or AZ) is a single network failure domain that hosts part or all of a Vault cluster. Examples of availability zones include:- an individual datacenter.- an air-gapped rack in a datacenter.- an availability zone in AWS or Azure.- a zone in GCP.
Region	A physical location in a distinct geographic area containing one or more availability zones.
Active node	The node in a Vault cluster that services write requests.

Term	Definition
Standby node	Nodes that forward write requests to the active node. They also service read requests in high availability mode. Standby nodes vote to elect a new active node if the current one is unavailable after an interval threshold.
Quorum	Quorum consists of a majority of voting nodes in a Vault cluster: `(N+1)/2`, where `N` = the total number of voting nodes. For example, if there are 3 voting nodes, we would need 2 nodes to form a quorum.Quorum ensures the integrity and consistency of secrets data. If a quorum of nodes is unavailable for any reason, the cluster becomes unavailable, meaning no client requests can be processed until quorum is once again achieved.
Redundancy zone (RZ)	Redundancy zones are a feature of integrated storage Autopilot. A redundancy zone consists of non-voting standby nodes deployed alongside voting nodes in a Vault cluster on a per-availability zone basis. This provides additional horizontal read scalability as well as increased fault tolerance for the Vault cluster. If a voting node fails, the non-voting node will be automatically promoted to a voter to preserve quorum.

Term	Definition
Disaster Recovery (DR) Replication	An advanced topology in which a secondary Vault Enterprise cluster replicates all data from its DR primary cluster. The DR secondary cluster does not service any requests from clients, but can be quickly promoted to an active cluster during a service interruption to the primary. They are warm standby clusters.
Cluster	Multiple Vault servers (nodes) operating together in a high-availability mode. Each node contains an identical set of replicated data storage.
Seal (mechanism)	The seal is a way of protecting the cryptographic material that Vault uses to encrypt and decrypt data. Depending on the mechanism for sealing Vault, the service returns an unseal key or recovery key to the operator. These keys are sharded by Shamir's secret sharing algorithm to ensure no single actor has access to all of the unseal or recovery material. The unseal or recovery key is reconstituted by entering a minimum threshold of key shares. Although the behavior of unseal keys and recovery keys differ, they both service the same ultimate goal: regaining access to the Vault cluster.

Term	Definition
Unsealing/Sealing (process)	Unsealing: Vault's storage is encrypted. The ability of the service to decrypt its secrets data is based on having the encryption key in memory. Vault is able to read the encryption key into memory indirectly by reconstituting the unseal/recovery key from key shares. The unseal key decrypts the encryption key.Sealing: When Vault deletes the encryption key from memory, the service is incapable of reading the encrypted storage. Vault will be unable to read it until a threshold of key shares reconstitutes the unseal/recovery key to decrypt the encryption key.
Tokens	Tokens are the core method for authentication within Vault.
Leases	An object in Vault containing metadata about a token's time duration and renewability. Vault creates leases for service tokens and dynamic secrets. Once the lease expires, Vault automatically revokes the data, and the consumer of the secret can no longer be certain that it is valid. For more information, see the HashiCorp Documentation on Leases.

1.5 Organizational requirements

Most tasks within this guide will be owned by a single team, which is responsible for deploying shared infrastructure services. This is sometimes referred to as a *platform team*. However, a successful deployment of Vault Enterprise requires collaboration across various functions within the organization. The platform team may interface with various other teams, such as networking, information security, or identity teams, to perform tasks such as allocating IP space, managing cer-

tificates, defining identity roles and permissions, or overseeing DNS hosted zones and records. In cases where the infrastructure resides on a public cloud, these functions might converge under a unified cloud team.

The primary platform team should thoroughly review this Solutions Design Guide in order to identify any teams they may be dependent on or may require approval from for the tasks required as part of the deployment. The platform team should appoint a project lead to oversee the deployment process and ensure effective communication with dependent teams and functions.

1.6 Guide relationship

This virtual machines track covers Vault Enterprise deployments on cloud infrastructure and private datacenters. For OpenShift deployments, use the OpenShift installation track. After installation, use the Vault Administration Guide for day-2 operations and the Vault User Guide for consumer workflows.

2 Architecture

In this section, we will explore the architecture of the HashiCorp Validated Design for Vault Enterprise. This architecture is designed to provide a highly-available, scalable, and secure Vault Enterprise deployment that is suitable for production workloads, and is intended to be used as a starting point for your own Vault Enterprise deployments. First, we will discuss the components that make up a single Vault cluster and its related infrastructure. We will then add a secondary Vault cluster, culminating in a multi-cluster architecture that can be used to provide both high-availability as well as disaster recovery for your Vault Enterprise deployment.

The primary components that make up a single Vault cluster are:

- Cluster of Vault nodes
- Load balancer
- Snapshot storage (backups)
- Seal mechanism
- Audit device(s)

2.1 Cluster

Vault uses a client/server architecture. The Vault server hosts an API, which Vault clients can communicate with via HTTP/HTTPS. Through client requests to the API, data is read from or written to the data storage backend.

2.1.1 Integrated storage

While Vault Enterprise has two data storage backends available, this architecture uses the integrated storage backend. integrated storage is based on the Raft consensus algorithm, and is built directly into the Vault binary. Because of this, the API and storage layers of Vault run on the same machine. There is no need to install additional software in order to use the integrated storage backend.

2.1.2 Clustering

Vault supports multi-node deployment for high-availability. This mode protects against outages by configuring multiple Vault nodes to operate as a single Vault cluster. To ensure this high-availability, nodes within a cluster should always be spread across multiple availability zones.

When using integrated storage, the cluster is bootstrapped during the initialization process. Initial bootstrapping results in a cluster of 1 node. This node is the *active node* (also known as the *leader*). Additional nodes can then be joined to the active node as *standby nodes* (also known as *followers*). Once a standby node has successfully joined the cluster, data from the active node will begin to replicate to it.

In a Vault cluster, only the active node performs write operations against the storage backend. Standby nodes can perform reads against storage and forward writes to the active node. This functionality is called *performance standby mode*, and is enabled by default for all standby nodes when using integrated storage on Vault Enterprise.

A Vault cluster must maintain quorum in order to continue servicing requests. Because of this, nodes should be spread across availability zones to increase fault tolerance.

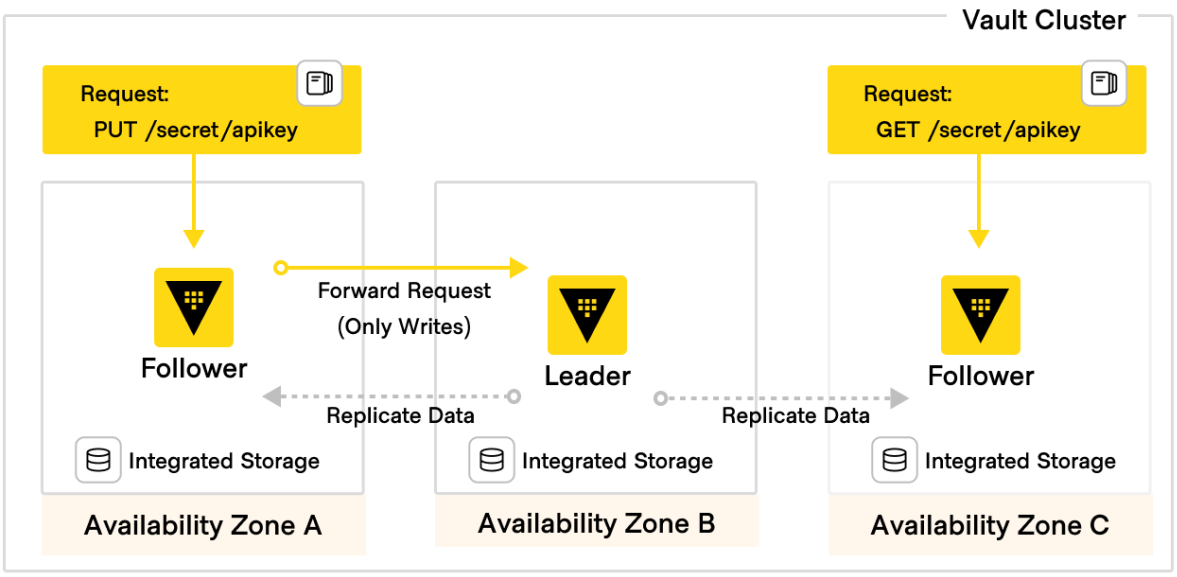


Figure 1: HVD Vault HA 3 nodes diagram

Voting nodes vs non-voting nodes

By default, standby nodes are also voters. If the active node becomes unavailable, a new active node is chosen by and among the remaining standby nodes. However, standby nodes can also be specified as *non-voters*. Non-voting nodes also act as data replication targets, but do not contribute to the quorum count.

Autopilot

A non-voting node can be be promoted to a voting node through the use of Vault Enterprise's Autopilot functionality. In addition to this, autopilot also enables several automated workflows for managing integrated storage clusters, including server stabilization, automated upgrades, and redundancy zones. For more information, see the HashiCorp documentation on Vault Enterprise Autopilot.

Redundancy zones

Using Vault Enterprise Redundancy Zones, the Vault cluster leverages a pair of nodes in each availability zone, with one node configured as a voter and the other as a non-voter. If one voter becomes

unavailable (regardless of whether it is an active node or a standby node), autopilot will promote its non-voting partner to take its place as a voter. This architecture gives high availability both within an availability zone as well as across availability zones, ensuring that `(n+1)/2` redundancy is maintained at all times.

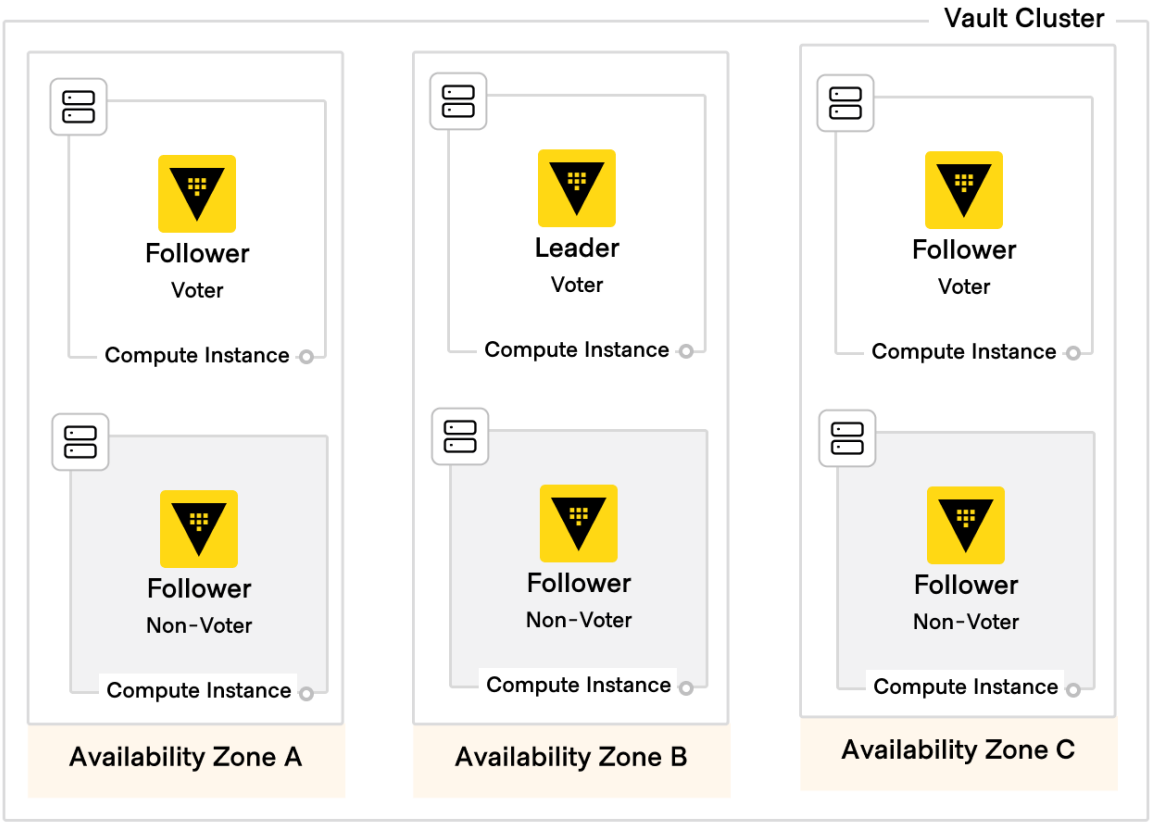


Figure 2: HVD Vault HA 6 nodes redundancy zones diagram

2.2 Load balancing

Use a layer 4 load balancer capable of handling TCP traffic to direct Vault client requests to the server nodes within your Vault cluster. Using a load balancer reduces the amount of static configuration on Vault clients, as they will access the service via the load-balanced endpoint, rather than sending requests directly to an individual node. Any request sent to a standby node that requires an active node to process (e.g. write requests) will be proxied by the standby node to the active node.

Vault server nodes also perform server-to-server communication to other nodes in the cluster over `8201/tcp`. This is used for functions critical to the integrity of the cluster itself, such as data replication or active node election. Unlike client requests, server-to-server traffic should *not* be load-balanced. Instead, each node must be able to communicate directly with all other nodes in the cluster.

Tip

Vault also automatically enforces encrypted communication between Vault nodes via an additional port separate from client communication. This communication is internal to Vault and does not require additional certificates or management.

2.2.1 TLS

To ensure the integrity and privacy of transmitted secrets, communications between Vault clients and Vault servers should be secured using TLS certificates that you trust. Do not manage the TLS certificate for the Vault cluster on the load balancer. Instead, the certificate chain (i.e. a public certificate, private key, and certificate authority bundle from a trusted certificate authority) should be distributed onto the Vault nodes themselves, and the load balancer should be configured to pass through TLS connections to Vault nodes directly. Terminating TLS connections at the Vault servers offers enhanced security by ensuring that a client request remains encrypted end-to-end, reducing the attack surface for potential eavesdropping or data tampering.

Warning

Avoid using self-signed certificates for production Vault workloads.

2.3 Backup snapshot storage

Automated snapshots should be saved to a storage medium, separate from the storage used for the Vault storage backend. Snapshot storage does not require the same high IO-throughput requirements as the rest of Vault, but it should be durable enough to ensure fast retrieval and restoration of backups when needed. To satisfy this requirement, use an object storage system such as AWS S3, or a network-mounted filesystem, such as NFS. Configuring the snapshots themselves, their frequency, and other operational considerations can be found in Vault: validated design guidance.

2.4 Sealing and unsealing

In order to protect itself and your data, Vault will *seal* itself under certain conditions, such as during a loss of quorum.

Sealing prevents control of Vault from falling into the hands of a single actor. By default, when Vault becomes sealed, it can be unsealed by manually submitting enough *unseal key shards*, which are created when the Vault cluster is first initialized.

Because this process is manual, it can become suboptimal when you have many Vault clusters, as there are now many different key holders with many different keys. For Vault Enterprise deployments, we instead recommend configuring an *auto-unseal mechanism*. Auto-unseal eliminates the need for manual intervention, and reduces the operational complexity of keeping Vault unseal keys secure.

2.4.1 Auto-Unseal using cloud-based KMS services

The most common method of auto-unsealing is through the use of a cloud-based key management service (KMS) system, such as AWS KMS, Azure Key Vault, or GCP Cloud KMS. For more information, refer to the HashiCorp Documentation on Auto Unseal.

Warning

Important note About recovery keys:Recovery keys are purely an authorization mechanism, and cannot decrypt the root key. This means that recovery keys are not sufficient to unseal Vault if the auto unseal mechanism is not working. Using auto-unseal creates a strict Vault lifecycle dependency on the underlying seal mechanism. If the seal mechanism (such as the cloud KMS key) becomes unavailable, or is deleted before the seal is migrated, there is no ability to recover access to the Vault cluster until the seal mechanism becomes available again. If the seal mechanism or its keys are permanently deleted, then the Vault cluster cannot be recovered, even from backups. To mitigate this risk, we recommend careful controls around management of the seal mechanism, such as AWS Service Control Policies.

If your architecture will include any secondary Vault clusters, configure these clusters with a *different* seal mechanism than that of the primary. This ensures that if the primary cluster becomes unrecoverable due to an issue with its seal mechanism, the secondary cluster can still be accessed.

Regardless of the seal mechanism you choose to deploy with your Vault cluster, we recommend encrypting the key shares and initial root token value at initialization time with user-supplied public keys. Operators can generate these keys from OpenPGP-compliant software such as GNU Privacy Guard (GPG).

2.4.2 Auto-unseal using a hardware security module

As an alternative to cloud-based KMS, you can use a hardware security module (HSM) to provide a secure auto-unsealing method for Vault. This method is primarily recommended for certain private datacenter environments in which the use of cloud-based KMS services is not available or not suitable for the deployment. This method enables Vault to use HSM devices (supporting PKCS11 operations) to encrypt the root key. Vault stores this HSM-encrypted root key in internal storage, and the HSM device automatically decrypts it.

HSM automatic unsealing with Vault involves using the HSM to encrypt and store Vault's unseal key. Upon Vault startup, instead of manually entering unseal keys, the unseal key is automatically decrypted by the HSM. This process ensures that the unsealing procedure is both secure and automated, reducing the risk of human error.

In addition to the unseal key managed by the HSM, Vault will also generate recovery keys to authorize human recovery operations on the Vault cluster. Because a quorum of recovery key shares can be used to generate a new root token, and because root tokens can grant extensive access to Vault, PGP encryption should always be used to protect the returned recovery key shares. This ensures that even if recovery keys are exposed, they remain encrypted and unusable without the corresponding PGP key.

Both rekeying the root key and rotation of the underlying data encryption key are supported when using an HSM.

2.5 Audit devices

Audit devices collect and keep a detailed log of all Vault requests and corresponding responses. Vault operators can use these logs to ensure that Vault is being used in a compliant manner, and to provide an audit trail for forensic analysis in the event of a security incident.

To ensure the availability of audit log data, configure Vault to use *multiple* audit devices. Writing an additional copy of audit log data to a separate device can help prevent data loss in the event one

device becomes unavailable, and can also help guard against data tampering. More information on the configuration and management of audit devices and audit logs can be found in Vault: validated design guidance.

2.6 Cluster architecture summary

To summarize, the HashiCorp Validated Design for a Vault Enterprise cluster architecture is made up of the following components:

- 6 Vault servers, deployed across three availability zones within a geographic region.
- Using a redundancy zones topology, each node is configured as a voter or non-voter, with one of each per availability zone.
- Locally-attached, high-IO storage for Vault's integrated storage.
- A layer 4 (TCP) load balancer, with TLS certificates distributed to the Vault nodes.
- An object storage or network filesystem destination for storing automated snapshots.
- An auto-unseal mechanism, such as a cloud-based KMS or an HSM.
- Audit devices, configured to write to multiple destinations.

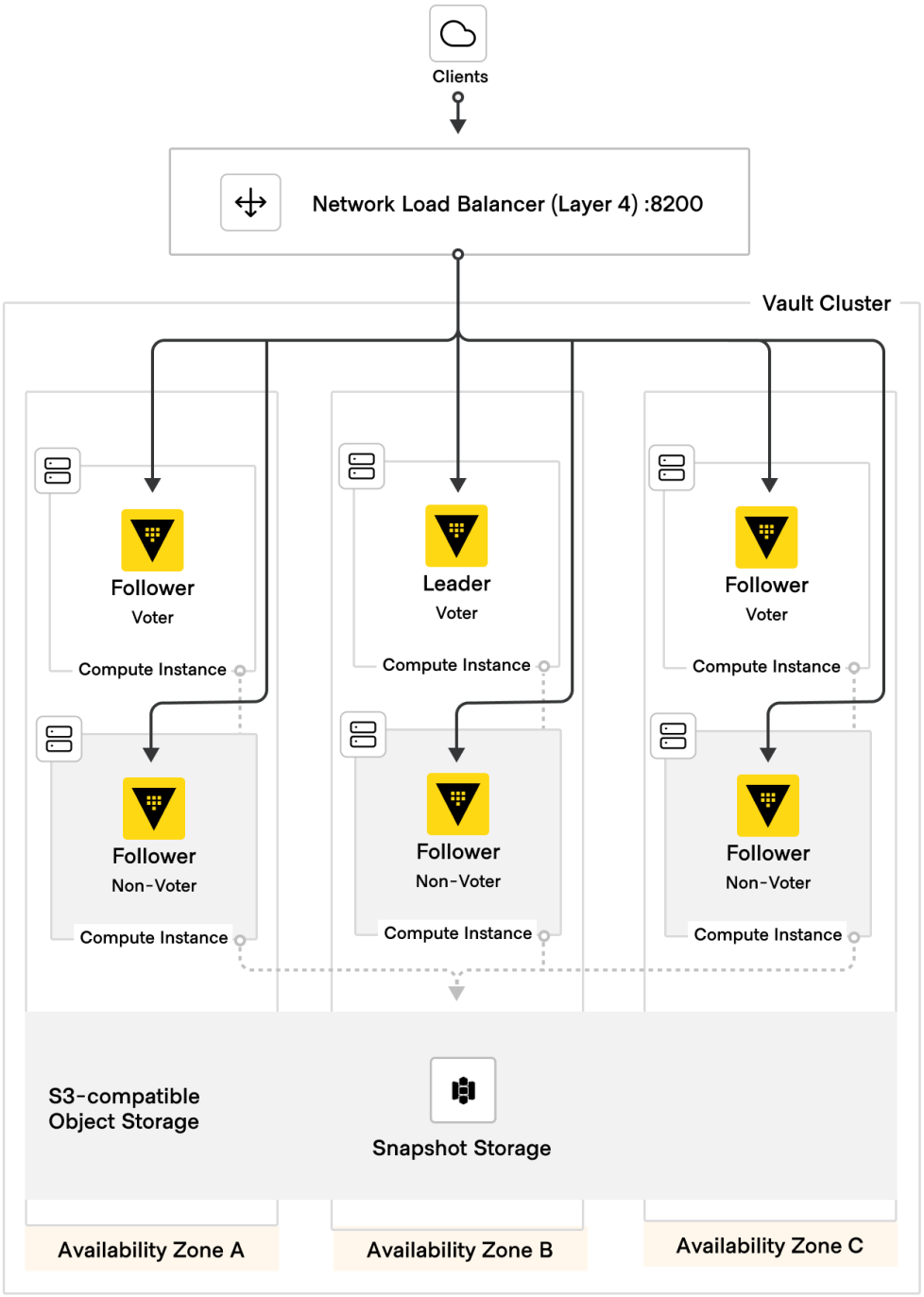

Clients
Network Load Balancer (Layer 4) :8200
Vault Cluster
Follower
Voter
Compute Instance
Leader
Voter
Compute Instance
Follower
Voter
Compute Instance
Follower
Non-Voter
Compute Instance
Follower
Non-Voter
Compute Instance
Follower
Non-Voter
Compute Instance
S3-compatible
Object Storage
Snapshot Storage
Availability Zone A
Availability Zone B
Availability Zone C

Figure 3: HVD Vault cluster architecture diagram

2.7 Disaster recovery secondary cluster

Disaster recovery (DR) secondary clusters are essentially warm standbys of the primary Vault cluster, and are used to provide continuity of service in the event of a cluster-wide outage. They do not service any of their own requests, but instead replicate data from the primary cluster. The replicated data not only includes secrets, but also tokens and leases.

If the primary cluster becomes unavailable, Vault operators can manually fail over to the DR secondary cluster by promoting it to become the primary. This procedure restores access to the Vault service and its secrets data, so that clients can reconnect to Vault without having to re-authenticate.

DR secondary clusters share the same fundamental architecture as primary clusters, including related components such as network load-balancing, seal mechanisms, and audit devices. DR secondary clusters should always be equally sized in order to accept the same volume of requests as the primary.

DR secondary clusters are typically placed in different geographical regions than their primary clusters to guard against region-level outages, though this is not a requirement. There must be network connectivity between these regions to allow for continuous data replication.

Automated snapshots cannot be configured for DR secondary clusters before promotion. Vault operators can instead take manual snapshots if desired.

The DR secondary cluster should also use auto-unseal, encrypted with a KMS key different from the one used on the primary cluster. After replication is enabled on the DR secondary cluster, the primary's keyring (which contains the recovery keys) are copied to the DR secondary as part of the bootstrapping process. If the secondary cluster also uses KMS auto-unseal, it utilizes the primary cluster's recovery keys and configuration (e.g., number of fragments and key share threshold). This can be changed with manual configuration on the DR secondary cluster. For more information, see the HashiCorp Documentation on Vault Enterprise Replication and Vault Replication Internals.

2.7.1 Failover mechanism

The promotion of a DR secondary cluster to a primary cluster is a manual process.

The process of promoting and demoting Vault clusters is a non-trivial operation. If Vault briefly encountered service unavailability and the failover process was automated, this could cause unintentional data loss and/or data consistency issues.

Vault operates in an active-passive configuration, meaning that only one cluster is the source of truth for all shared secrets in an environment. This mitigates the split-brain problem in which multiple Vault clusters could separately manage the same secrets. To ensure that one cluster is always the source of truth, the standard DR failover promotes a DR cluster to serve all client requests. The demoted cluster does not attempt to immediately reconnect to a DR primary cluster, but maintains its current dataset so that Vault operators can reconnect it and update it later.

Because DR secondary clusters do not service any client requests, they only become active when operators promote DR replicas and failover client requests. However, clients should not need to reconfigure how they communicate with Vault. Use an additional routing mechanism that can route clients to the active primary cluster, regardless of region. Examples of this include, but are not limited to:

- HashiCorp Consul
- Amazon Route53 DNS Failover
- Azure Traffic Manager
- Google Cloud DNS with HTTP(S) Load Balancer
- F5 Global Server Load Balancing (GSLB)

The following diagram illustrates the complete multi-cluster architecture as described above:

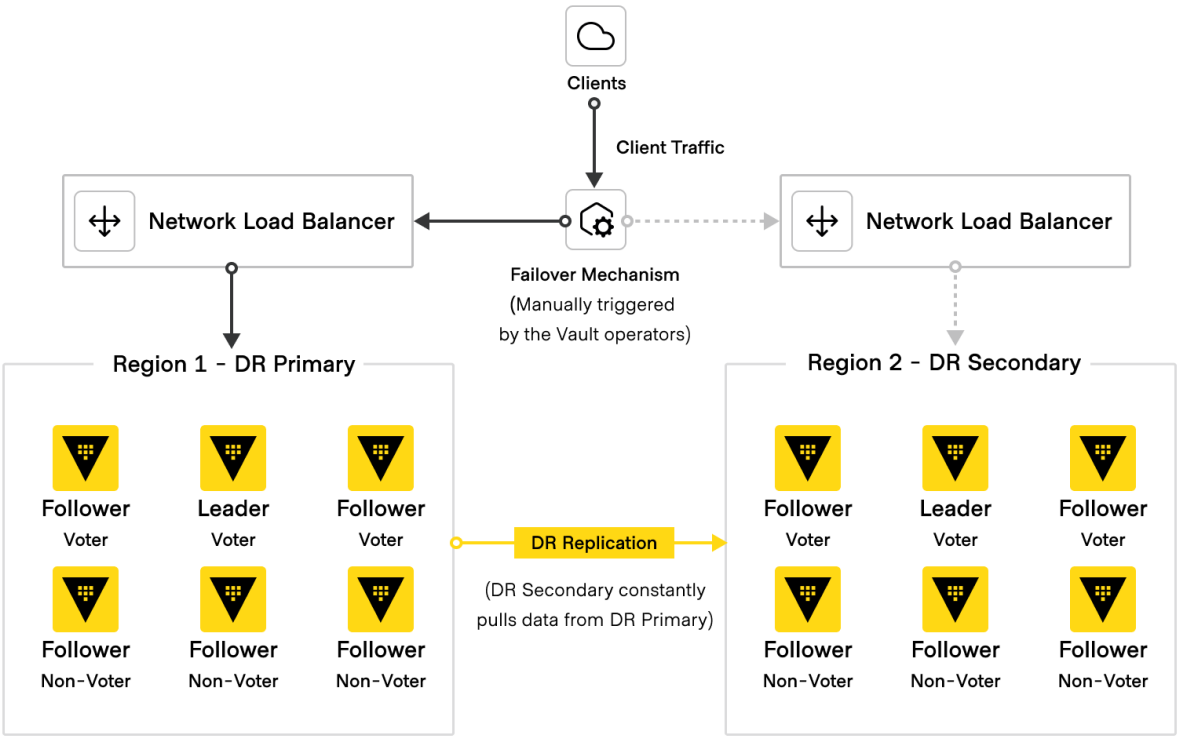


Figure 4: HVD Vault Disaster Recovery Diagram

We will discuss each of these components and their configuration in more detail in the next section.

3 Detailed design

3.1 Core design requirements

This section takes the recommended architecture from the preceding section and provides more detail on each component of the architecture. Review this section to identify all technical and personnel requirements before moving on to implementation.

3.1.1 Compute

This section contains specific hardware capacity recommendations, network requirements, and additional infrastructure considerations.

Sizing

Every hosting environment is different, and every customer's Vault usage profile is different. These recommendations only serve as a starting point, from which each customer's operations staff may observe and adjust to meet the unique needs of each deployment. For more information, refer to the Vault Reference Architecture section on Sizing For Vault Servers.

This guide divides sizing recommendations into two common cluster sizes: small, and large. Small clusters are appropriate for most initial production deployments or for development and testing environments. Large clusters are production environments with a consistently high workload. That might be a large number of transactions, a large number of secrets, or a combination of the two.

Size	CPU	Memory	Disk capacity	Disk IO	Disk throughput
Small	2-4 cores	8-16 GB RAM	100+ GB	3000+ IOPS	75+ MB/s
Large	4-8 cores	32-64 GB RAM	200+ GB	10000+ IOPS	250+ MB/s

For a mapping of these requirements to specific instance types for each cloud provider, refer to the cloud-focused sections below.

Hardware considerations

In general, CPU, and storage performance requirements depend on the customer's exact usage profile (for example, types of requests, average request rate, and peak request rate). Memory requirements depend on the total size of data stored in memory. Size memory according to that data.

Note

For predictable performance on cloud providers, avoid burstable CPU and storage options, such as AWS `t2` and `t3` instance types, Azure B-series VMs, or GCP e2-micro/small instances. Performance for these types of instances may degrade under continuous load.

When using integrated storage, Vault servers must have a high-performance hard disk subsystem capable of up to 10000+ IOPS and 250+ MB/s disk throughput. Many vault operations (such as login requests) require multiple disk writes. When the system generates or rotates many secrets frequently, Vault also needs to flush this data to disk. In both cases, the use of slower storage systems negatively impacts performance.

HashiCorp strongly recommends configuring Vault with audit logging enabled, as well as telemetry for diagnostic purposes. Vault: validated design guidance covers the configuration of these components. The impact of the additional storage I/O from audit logging varies depending on your particular pattern of requests.

Tip

Always write audit logs to a separate disk. This approach reduces the risk of performance bottlenecks that can occur when audit logs compete for resources with other system processes.

Warning

Configure multiple audit devices to enhance reliability and redundancy. If one device encounters issues, others continue to function, ensuring continuous, uninterrupted logging. This setup is crucial for maintaining robust audit trails and supporting both effective troubleshooting and security monitoring. Vault Enterprise stops operations if all audit devices fail.

3.1.2 Networking

Vault requires node interconnection between availability zones (AZs in AWS/GCP, or Availability Zones in Azure), client redirect logic, and load balancers configured with TLS passthrough to connect clients to clusters.

> **Note**
>
> For security reasons, do not expose Vault clusters to the public internet.

Node interconnectivity requires low latency (around 8 ms) to replicate secrets across the integrated storage backend, as well as to efficiently route client write requests from standby nodes to the active node. These nodes are also fronted by Layer 4 load balancers, which permit end-to-end TLS communication.

Network rules must allow Vault to communicate across TCP ports designated for API and intracluster traffic. Deploy Vault nodes in private subnets. Deploy network address translation (NAT) devices in public subnets to allow VMs/instances to retrieve critical security updates through egress Internet traffic.

Availability zones

This Validated Design uses 3 availability zones, with 2 nodes per availability zone.

In order for cluster members to stay properly in sync, we recommend network latency between availability zones be less than eight milliseconds (8 ms).

Network connectivity

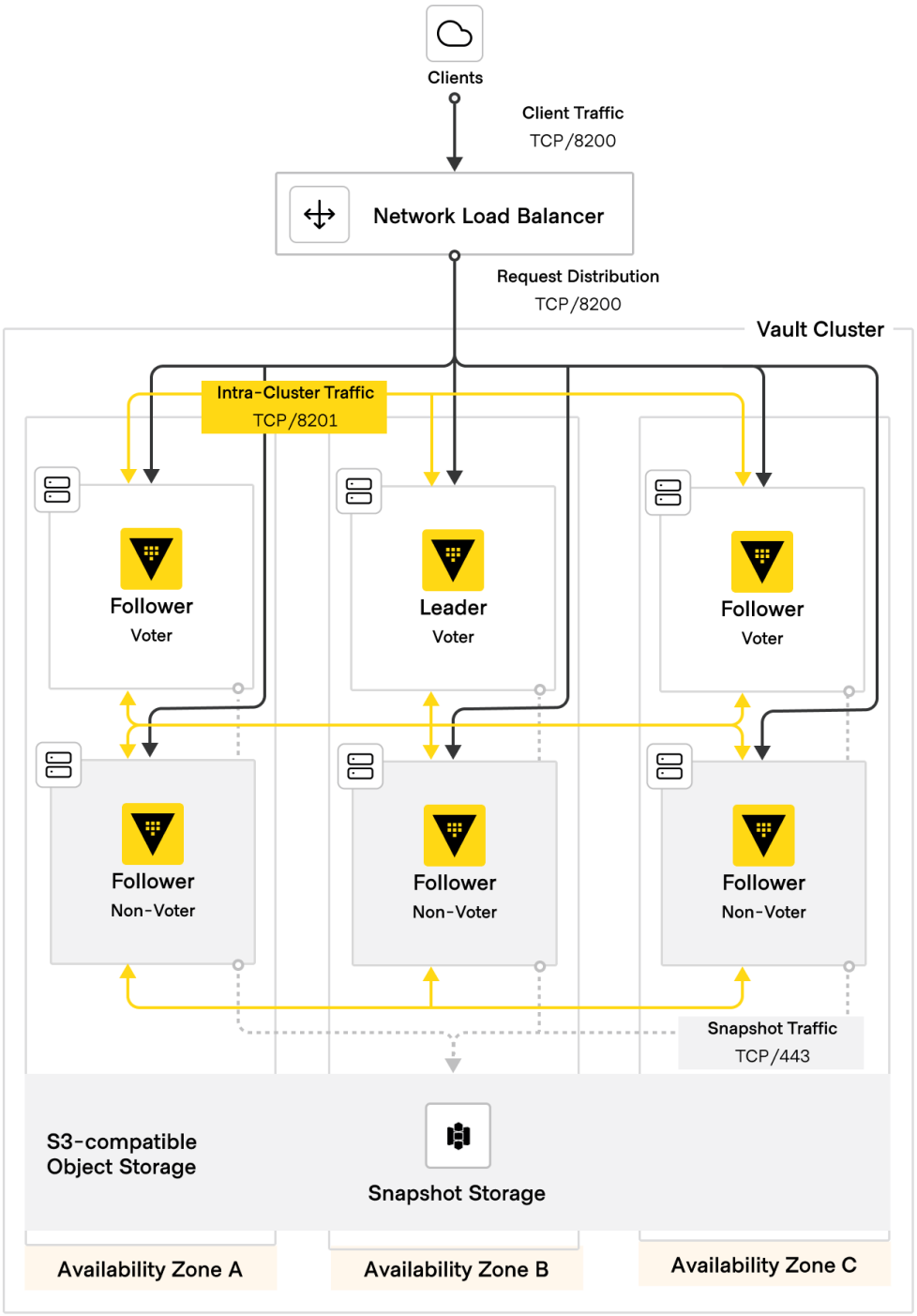


Figure 5: HVD Vault networking diagram

The following table outlines the minimum network connectivity requirements for Vault cluster nodes. You may also need to grant the Vault servers outbound access to additional services that live elsewhere, either within your own internal network or through the Internet. Examples may include:

- Authentication and secret provider backends, such as cloud providers (AWS, Azure, GCP) or a Kubernetes cluster.
- Network Time Protocol (NTP), for maintaining consistent time between nodes.
- Remote log handlers, such as a Splunk or ELK environment.
- Metrics collection, such as Prometheus or Datadog.

Source	Destination	Port	Protocol	Direction	Purpose
Client machines/administration	Load balancer	8200	tcp	incoming	Request distribution
Load balancer	Vault servers	8200	tcp	incoming	Vault API; `api_addr`
Vault servers	Vault servers	8200	tcp	bidirectional	Cluster bootstrapping
Vault servers	Vault servers	8201	tcp	bidirectional	Raft, replication, request forwarding; `cluster_addr`
Vault servers	External systems	various	various	various	External APIs

Do not expose Vault to inbound traffic through a publicly accessible endpoint. This minimizes both the risk of exposing secrets and the risk of denial-of-service attacks. Wherever possible, limit access to the Vault API to the networks where you need to access Vault.

Here is an overview of an example write transaction to Vault:

1. Clients make HTTPS requests to Vault through the load balancer endpoint over port `8200`.

2. The load balancer selects a target and forwards the request to one of the Vault servers in the cluster through port `8200`.
3. The Vault server that receives the request terminates TLS.
4. If the node that receives the request is not the active node, the request is then forwarded from the standby node to the active node through port 8201 using the IP address known from the `config.hcl`.
5. The active node receives the request and performs the write operation.
6. The node that received the request sends a response to the client. This happens regardless of whether it is the active node – clients are not aware of whether the system forwarded the request.
7. All data written to the active node is then replicated to its standby nodes through port `8201`.

> Note
>
> The amount of network bandwidth used by Vault depends entirely on the specific customer's usage patterns. In many cases, even a high request volume does not translate to a large amount of network bandwidth consumption. It is also important to consider bandwidth requirements to other external systems, such as monitoring and logging collectors.

Traffic encryption

Encrypt every segment of Vault-related network traffic in transit using TLS over HTTPS. This includes communication from clients to the Vault servers, as well as communication between cluster members.

Create a standard X.509 public certificate from an existing trusted certificate authority internal to your organization. This certificate represents the Vault cluster, and each Vault server in the cluster serves it. Along with the certificate itself (`vault-public.pub`), also supply the certificate's private key (`vault-private.key`) and a bundle file from the certificate authority used to generate the certificate (`ca.pub`). The implementation section of this document references these files.

Vault also encrypts server-to-server communication, which requires no additional configuration. Vault automatically negotiates an mTLS connection between servers when new members join the cluster.

Domain name system

Internally, Vault nodes do not use DNS names to talk to other members of the cluster, relying instead on IP addresses. However, you need to configure a DNS record that resolves to the IP address hosting your load balancer configuration. This guide uses `vault.domain` as the example value to refer to this DNS record in later sections.

Load balancing

Use a TCP-based (layer 4) load balancer to distribute incoming requests across multiple Vault nodes within a cluster. Configure the load balancer to listen on port `8200/tcp` and forward requests to all Vault nodes in the cluster on port `8200/tcp`. *All* healthy nodes in the cluster must be able to receive requests in this way, including standby nodes (that is, followers) and nodes configured as non-voters.

> **Tip**
>
> While this guide uses the default port `8200/tcp` for the load balancer, using another port (such as `443/tcp`) is also supported, and is a common practice for Vault deployments.

TLS passthrough

Configure the load balancer to use TLS passthrough, rather than terminating the TLS connection at the load balancer. This keeps the traffic end-to-end encrypted from the client to the Vault nodes, removing even the load balancer as a possible attack vector.

Health check

Each Vault node provides a health check API endpoint at `/v1/sys/health`. HTTPS-based health checks provide a deeper check than checking the TCP port.

Configure your load balancer to perform a health check to all Vault nodes in a cluster by using the path `/v1/sys/health?perfstandbyok=true`, with an expected status code of `200`. By default, only a leader node returns a `200` status code. Setting the `perfstandbyok` parameter to **`true`** instead directs all nodes to return a `200` status code, including non-leader nodes (also known as performance standby nodes). Configuring the health check in this way enables the load balancer to spread client read requests to all nodes in the cluster, while the system still redirects write requests sent to performance standby nodes to the leader node using request forwarding.

For specific details on all query options and response codes and their meanings, refer to the HashiCorp Vault Documentation on the `/sys/health` API endpoint.

> **Tip**
>
> If possible, use a load balancer that supports both TLS passthrough and HTTPS-based health checks. AWS Network Load Balancer, Azure Load Balancer, GCP Network Load Balancer, F5 BigIP, Citrix application delivery controller (ADC), and HAProxy are just a few examples of load balancers that support this combination of capabilities.Some load balancers, such as Nginx, do not allow for both TLS passthrough and the use of HTTPS-based health checks. If you cannot use another type of load balancer in your environment, you can still use a port-based check instead. Please note that port-based health checks are a more limited type of check, and may not detect issues that an HTTPS-based check detects.

3.1.3 Software

To deploy production Vault, this Validated Design only requires that you install the Vault Enterprise software binaries on the Vault servers.

Do not run the Vault process as root. Instead, create a dedicated user to run the Vault service and to protect the files used by and created by Vault.

Memory locking and swap configuration

When using integrated storage, you must explicitly configure the `disable_mlock` parameter in your Vault configuration file. Vault does not start without this explicit configuration.

> **Note**
>
> For details on the `disable_mlock` parameter, refer to the Vault configuration documentation.

We recommend you disable `mlock` when using integrated storage. The `mlock()` system call does not interact well with memory-mapped files such as those created by BoltDB, which Raft uses to track state. When you enable `mlock`, memory-mapped files get loaded into resident memory, causing Vault to load its entire dataset in-memory. This can cause out-of-memory issues if Vault's data becomes larger than the available RAM.

We recommend you disable swap. Even though the data within BoltDB remains encrypted at rest, disable swap on the host system to prevent Vault from writing other in-memory sensitive data to disk.

This approach prioritizes system stability while maintaining security through proper host hardening. If an attacker has access to read swap space, they likely already have access to read live memory, making host-level security controls the critical defense layer.

If you choose not to disable `mlock` (not recommended for integrated storage), you must grant the Vault user access to the `mlock()` system call. For systemd-based OSes, you can specify this capability in the Vault systemd unit file:

```
[Unit]
Description=Vault service
Requires=network-online.target
After=network-online.target

[Service]
User=vault
Group=vault
ExecStartPre=/sbin/setcap 'cap_ipc_lock=+ep' $(readlink -f $(which vault))
ExecStart=/usr/local/bin/vault server -config /etc/vault.d/config.hcl
Capabilities=CAP_IPC_LOCK+ep
CapabilityBoundingSet=CAP_SYSLOG CAP_IPC_LOCK
...
```

This configuration is **not recommended** when using integrated storage due to the memory issues described in the preceding section.

Vault license

Deploy your Vault Enterprise license on every Vault node for the Vault process to start. The implementation section has more details on how to autoload the license at each Vault start time.

3.1.4 Storage

Data storage

Use integrated storage as the storage backend for Vault. Vault's persistent data storage lives on the Vault servers.

Backups

Integrated storage provides an Automated Snapshots feature, which enables you to perform backups of your data within Vault. Unlike OS- or volume-based snapshots, Vault automated snapshots only capture the integrated storage dataset. This ensures data integrity and consistency when recovering Vault.

To store your snapshots, you can either use an object store, such as Amazon S3, Azure Blob Storage, or Google Cloud Storage, or a durable remote filesystem, such as NFS. Vault: validated design guidance details the procedure for configuring automated snapshots.

3.1.5 Configuration

This section covers the various sections of your Vault configuration file. The Vault server process references this file each time it starts.

The configuration file contains a number of different stanzas. Below you find required fields for each stanza. In some cases the fields contain the recommended values, but others depend on your particular environment. In those cases, take note of what the relevant values are, as you need them during the installation process as detailed in the Implementation section.

`listener` stanza

This stanza defines the listener configuration for Vault. It specifies the address and port on which Vault listens for incoming connections, as well as the TLS certificate and key files to use for TLS termination.

```
listener "tcp" {
  address       = "0.0.0.0:8200"
  tls_cert_file = "/etc/vault.d/server.crt"
  tls_key_file  = "/etc/vault.d/server.key"
  tls_client_ca_file = "/etc/vault.d/ca.pub"
}
```

`tls_cert_file` specifies the path to the server's TLS certificate file. `tls_key_file` specifies the path to the server's private key file. `tls_client_ca_file` specifies the path to the certificate authority file, which the system uses to verify client certificates if you want to enforce client-side authentication.

Tip

The CA file listed in `tls_client_ca_file` is only used as part of client-based authentication, and it is not used to validate the web server itself.

Tip

If you wish to include an issuer CA certificate as part of the certificate chain, you can concatenate the certificate into `tls_cert_file`.

`storage` stanza

This stanza defines integrated storage (also known as `raft`) as the Vault storage method, defines where Vault stores data on disk for each server, and lists the other servers that belong to the cluster.

```
storage "raft" {
  path    = "/var/lib/vault/data"
  node_id = "vault1"

  retry_join {
    leader_api_addr         = "https://vault-1.domain:8200"
    leader_ca_cert_file     = "/etc/vault.d/ca.pub"
    leader_client_cert_file = "/etc/vault.d/vault-public.pub"
    leader_client_key_file  = "/etc/vault.d/vault-private.key"
    leader_tls_servername   = "vault.domain"
  }

  retry_join {
    leader_api_addr         = "https://vault-2.domain:8200"
    leader_ca_cert_file     = "/etc/vault.d/ca.pub"
    leader_client_cert_file = "/etc/vault.d/vault-public.pub"
    leader_client_key_file  = "/etc/vault.d/vault-private.key"
    leader_tls_servername   = "vault.domain"
  }

  retry_join {
    leader_api_addr         = "https://vault-3.domain:8200"
    leader_ca_cert_file     = "/etc/vault.d/ca.pub"
    leader_client_cert_file = "/etc/vault.d/vault-public.pub"
    leader_client_key_file  = "/etc/vault.d/vault-private.key"
    leader_tls_servername   = "vault.domain"
  }

  ...
}
```

- `path`: Specifies the directory where Vault stores its data on disk.
- `node_id`: A unique identifier for the node within the Raft cluster. Can be any string.
- `retry_join`: This block configures Vault nodes to auto-join the Raft cluster on startup.

The `retry_join` stanzas identify other Vault nodes in the cluster. If you know the connection details for all nodes in the cluster in advance, you can use these stanzas to facilitate automatic joining of the nodes to the Raft cluster. Each stanza declares identifying details of a particular node. Upon initialization of one node as the leader, the remaining nodes use this configuration to locate and connect to the leader node, thus forming the cluster. For more information on the `retry_join` stanza, refer to the HashiCorp Vault documentation on Integrated Storage.

> **Tip**
>
> In practice, each node's configuration includes the same stanzas for simplicity. There is no need for a node to exclude itself in its configuration.

`seal` stanza

Your Vault config includes auto-unseal configuration using the `seal` stanza. Manual unsealing using the Shamir secret sharing algorithm does not require the use of this stanza.

Example of auto-unseal using AWS KMS:

```
seal "awskms" {
  region     = "us-west-2"
  kms_key_id = "abcd1234-a123-456a-a12b-a123b4cd56ef"
}
```

- `region` specifies the AWS region where the KMS key resides.
- `kms_key_id` specifies the ID of the AWS KMS key to use for auto-unsealing.

Example of auto-unseal using Azure Key Vault:

```
seal "azurekeyvault" {
  tenant_id      = "46646709-b63e-4747-be42-516edeaf1e14"
  vault_name     = "vault-auto-unseal-keyvault"
  key_name       = "vault-seal-key"
}
```

- `tenant_id` specifies the Azure Active Directory tenant ID.
- `vault_name` specifies the name of the Azure Key Vault.
- `key_name` specifies the name of the key in Azure Key Vault to use for auto-unsealing.

Example of auto-unseal using GCP Cloud KMS:

```
seal "gcpckms" {
  project     = "my-project"
  region      = "us-east1"
  key_ring    = "vault-keyring"
  crypto_key  = "vault-key"
}
```

- `project` specifies the GCP project ID.
- `region` specifies the GCP region where the Cloud KMS key resides.

- `key_ring` specifies the name of the Cloud KMS key ring.
- `crypto_key` specifies the name of the crypto key to use for auto-unsealing.

Example of auto-unseal using PKCS11 (HSM):

```
seal "pkcs11" {
  lib            = "/usr/vault/lib/libCryptoki2_64.so"
  slot           = "2305843009213693953"
  pin            = "AAAA-BBBB-CCCC-DDDD"
  key_label      = "vault-hsm-key"
  hmac_key_label = "vault-hsm-hmac-key"
}
```

- `lib`: Path to the Public Key Cryptography Standards (PKCS) #11 library on the virtual machine that installs Vault Enterprise.
- `slot`: The slot number to use.
- `pin`: PKCS #11 personal identification number (PIN) for login.
- `key_label`: Defines the label of the key you want to use.
- `hmac_key_label`: Defines the label of the key you want to use for hash-based message authentication code (HMAC).

PKCS #11 authentication occurs using a slot number and personal identification number (PIN). In practice, because the PIN is not required to be numeric (and some hardware security modules (HSMs) require more complex PINs), this behaves like a username and password.

Like a username and password, protect these values. If you store them in Vault's configuration file (like the stanza example in the preceding section), control read access to the file for appropriate users.

Rather than storing these values into Vault's configuration file, you can also supply them using environment variables.

> **Note**
>
> Your HSM's configuration may require additional fields, depending on the device. Consult the Vault HSM seal documentation and your HSM documentation as needed.

`telemetry` stanza

The Vault server process collects various runtime metrics about the performance of different libraries and subsystems. Stream and store telemetry from Vault in metrics aggregation software

to monitor Vault and collect durable metrics. For more information, see the HashiCorp Vault documentation on Telemetry.

The `telemetry` stanza defines how metrics become available to your telemetry platform. If you do not already have an existing metrics platform, we recommend using Prometheus, which has a good balance of simplicity of configuration and scalability.

Example of a telemetry configuration using Prometheus:

```
telemetry {
  disable_hostname = true
  prometheus_retention_time = "12h"
}
```

- `prometheus_retention_time` (string: "24 h"): Specifies the amount of time that Prometheus retains metrics in memory. Setting this to `0` disables Prometheus telemetry.
- `disable_hostname` (boolean): prevents the system from prefixing metrics with hostname.

Warning

By default, the system scrapes metrics from the leader node, which may change over the lifetime of a cluster. If `disable_hostname` is not set to true, the system prefixes metrics with the hostname of the leader node, which causes the system to lose metrics when the leader changes.

You can find additional guidance about collecting metrics using Prometheus in Vault: validated design guidance. For more information, see the HashiCorp Vault documentation on Monitoring Telemetry using Grafana and Prometheus.

3.1.6 Unseal and recovery keys

Vault encrypts data with an encryption key. That encryption key is further encrypted with a second key, known as the root key. At Vault initialization time, you can choose to split the root key into a number of key shares using Shamir's secret sharing algorithm. When initializing Vault with the Shamir seal, the system calls generated key shares *unseal keys*. When initializing Vault with auto-seal or HSM, it knows the key shares as *recovery keys*. There are important differences between the two.

Both unseal keys and recovery keys can authorize Vault to perform certain operations, such as generating a new root key. However, only unseal keys are able to decrypt the root key and unseal

Vault. Recovery keys cannot decrypt the root key, and thus are not sufficient to unseal Vault if the Auto Unseal mechanism is not working. They are purely an authorization mechanism.

> **Warning**
>
> Using auto unseal creates a strict Vault lifecycle dependency on the underlying seal mechanism. This means that if the seal mechanism (such as the cloud KMS key) becomes unavailable or the system deletes it, then you cannot recover access to the Vault cluster until the mechanism is available again. If the system permanently deletes the seal mechanism or its keys, then you cannot recover the Vault cluster, even from backups.

For more information, see the HashiCorp Vault documentation on Seal Concepts.

Key share encryption

When you initialize Vault, it returns hexadecimal-encoded or base64-encoded representations of the key shares and initial root token value in plaintext by default. Vault can encrypt the key shares and initial root token value at initialization time with user-supplied public keys generated from any RFC 4880 compliant Pretty Good Privacy (PGP) software, such as GNU Privacy Guard (GPG).

By initializing Vault with the `pgp-keys` and `root-token-pgp-key` options, it

- Encrypts the unseal keys and root token value with the specified GPG public keys.
- Base64 encodes the encrypted values.
- Outputs those values instead of plaintext values.

Before installing Vault, choose the individuals who participate in this process. Each individual needs to create their own PGP-compliant public key to use in the encryption process. Please note that the system ties PGP encryption to the individual user and relies on their private key, which may not be well secured or reliably stored. You create these keys later as part of the implementation steps.

Key share threshold

Vault requires a threshold of unseal or recovery key shares to authorize certain vital operations in Vault. When generated, distribute these key shares amongst a *chosen set of trusted individuals*. However, consider the real-world implications of having humans responsible for providing key share data.

If a member of the team departs the organization or is otherwise severed, their portion of the recovery key becomes inaccessible if it is PGP encrypted. Over time, an organization likely experiences personnel attrition, thereby losing a threshold of key shares to perform vital recovery operations.

Operators plan in advance for how many key shares they want Vault to generate during initialization, and how many of those key shares meet the threshold requirement. Both options are configurable when initializing Vault. By default, Vault creates 5 key shares, with a threshold of 3 required to reconstitute the unseal or recovery key.

Rekeying and rotating Vault

In addition to configuring Vault to generate a higher number of recovery key shares, Vault operators also define a cadence by which they rekey Vault and rotate its encryption key. Rekeying is a manual process where you instruct Vault to regenerate the root key and its key shares. Common events that can warrant a rekey include:

- Someone joins or leaves the organization.
- Security wants to change the number of shares or threshold.
- Compliance mandates that you rotate the keys at a regular interval.

Rekeying the Vault requires a threshold number of unseal keys. Before continuing, ensure enough unseal key holders are available to assist with the rekeying to match the threshold configured when you issued the keys.

The standard Vault recommendation for unseal key shares/recovery key shares has historically been 5 key shares with a threshold of 3 to do sensitive actions. This recommendation comes from the competing priorities of making these actions straightforward to perform, but minimizing the likelihood that only one user can perform them without checks or balances. As long as you schedule a regular interval to rekey your keys and replace any keyholders that leave your team and rekey the keys at that point, this recommendation is likely sufficient for most customers, with some limited caveats.

Tip

If you are using a Shamir seal instead of an auto-unseal mechanism, you may wish to specify more key shares without raising your threshold. In the event that your Vault seals at an inconvenient time, you have a higher probability of paging your keyholders and unsealing Vault. If your Vault cluster has a large number of administrator teams managing it, and a large pool of administrators that work with it, you may want to examine having more keyshares, or the possibility of having a higher threshold if more teams need to validate sensitive actions.

In addition to rekeying the root key, you may independently want to rotate the underlying encryption key Vault uses to encrypt data at rest. Unlike rekeying the Vault, rotating Vault's encryption key does not require a quorum of unseal keys. Anyone with the proper permissions in Vault can perform the encryption key rotation.

3.2 Compute

Each Vault node in a cluster resides on a separate compute instance (EC2 instance in AWS, Virtual Machine in Azure, or Compute Engine instance in GCP). Shared or dedicated tenancy is acceptable. A single Vault cluster lives in a single region, with nodes deployed across all availability zones. This design leverages Vault Enterprise Redundancy Zones, and deploys six nodes in total across three availability zones. Within each zone, configure one node as a voting node and the other as a non-voting node.

3.2.1 Operating systems

The Validated Design Terraform module uses Ubuntu 22.04.

3.2.2 Sizing

AWS

On AWS, you can use `m5.2xlarge` or `m5.4xlarge` instance types to supply the recommended amounts of vCPU and RAM for Vault.Create new `gp3` volumes configured with desired disk throughput and IOPS for your expected workload. If you are unsure of your workload requirements, your HashiCorp account team can help you evaluate your use case and provide specific guidance. ::: {data-environment=tcolorbox data-environment-keyword="colback=HexColorFAFAFA,colframe=HexColorAEAEB2,coltitle=HexColor3B3D45,title=Tip"} You can also use instance types from later instance families, provided that they meet or exceed the preceding resource recommendations.

Azure

On Azure, you can use `Standard_D16s_v4` or `Standard_D32s_v4` VM sizes to supply the recommended amounts of vCPU and RAM for Vault.Create new `Premium SSD v2` managed disks, configured with the desired disk throughput and IOPS based on your expected workload. If you are unsure of your workload requirements, your HashiCorp account team can help you evaluate your use case and provide specific guidance. ::: {data-environment=tcolorbox data-environment-keyword="colback=HexColorFAFAFA,colframe=HexColorAEAEB2,coltitle=HexColor3B3D45,title=Tip"} You can also use newer VM sizes or families, provided that they meet or exceed the preceding resource recommendations.

:::

GCP

On GCP, you can use `n2-standard-8` or `n2-standard-16` instance types to supply the recommended amounts of vCPU and RAM for Vault.Create new `pd-ssd` persistent disks, configured with desired disk throughput and IOPS for your expected workload. If you are unsure of your workload requirements, your HashiCorp account team can help you evaluate your use case and provide specific guidance. ::: {data-environment=tcolorbox data-environment-keyword="colback=HexColorFAFAFA,colframe=HexColorAEAEB2,coltitle=HexColor3B3D45,title=Tip"} You can also use instance types from newer machine families, provided that they meet or exceed the preceding resource recommendations.

:::

:::

3.2.3 Autoscaling

Note

The preceding architecture design is for ease of deployment and self-healing, but not for scaling. Vault itself is not natively elastic.

3.3 Networking

3.3.1 Private network

The Validated Design module creates a new virtual network (VPC in AWS/GCP, or VNet in Azure) and deploys a number of resources:

- One private and one public subnet for each availability zone (total of 6).
- Private and public route tables.
- One NAT gateway per subnet, each paired with a public IP (total of 3 NAT gateways and 3 public IPs).
- One Internet gateway.
- Network security groups for ingress and egress (total of 2).

3.3.2 Subnets

The Validated Design deploys 3 private subnets and 3 public subnets across 3 availability zones. The Vault service is not exposed to the public Internet, so the module deploys VM instances into the private subnets.

The design deploys NAT Gateway instances in the public subnets. The design also deploys an Internet gateway into the virtual network.

3.3.3 Routing

Along with creating subnets, NAT gateways, network security groups, and load balancers, the Validated Design defines routes for connecting resources together. Virtual network route tables

- define ingress and egress routes among all of the private subnets for intra-node communication.
- Allow ingress and egress traffic from the private subnets to the NAT gateways (public subnets) for communication with the load balancer, as well as services such as Amazon S3, Azure Blob Storage, and Google Cloud Storage.
- Define the egress route between NAT gateways and the Internet gateway, thereby providing an Internet route for Vault nodes in private subnets.

3.3.4 Network security groups

The Validated Design does not deploy firewall rules or firewall configurations, but it does deploy network security groups (security groups in AWS/GCP, or Network Security Groups in Azure). These security groups facilitate instance communication within the Vault cluster and outside of it. One contains ingress-only rules for TCP ports 8200 and 8201, and the other is the virtual network default (unused).

3.3.5 Load balancing

For load balancing, use a Layer 4 network load balancer (Network Load Balancer in AWS, Azure Load Balancer, or Network Load Balancer in GCP) in front of all Vault cluster nodes, across all subnets.

There are load balancer listeners for TCP ports `8200` and `8201`. The target groups or backend pools associated with each of these listeners both contain the full set of all cluster nodes.

The system uses an HTTPS health check for all target groups using the path below.

```
/v1/sys/health?perfstandbyok=true&activecode=200&performancestandbycode=473
```

3.4 Software

3.4.1 Vault license

Deploy your Vault Enterprise license on every Vault node for the Vault process to start. When using the included Terraform modules, submit your license file as a variable to Terraform, which base64 encodes it and creates a new secret with its contents on the respective cloud secrets management service (such as AWS Secrets Manager, Azure Key Vault, or Google Secret Manager). As part of the cloud-init process, the system reads the license contents from the secrets manager service, decodes them, and writes them to a file on the node at a location that the Vault service autoloads at start time.

3.5 Storage

When deploying on public cloud, use the native CSP cloud storage to store automated snapshots. Deploy the storage in the same region as the compute instances.

3.6 Auto-unseal

Operators deploying Vault in a cloud context leverage the key management service of the respective cloud (AWS KMS, Azure Key Vault, or GCP Cloud KMS) to generate the key used for auto-unsealing the cluster. The provided Terraform deployment modules create a customer managed key for this purpose, and reference the appropriate identifier for this key in the `seal` stanza of the Vault configuration file on each server.

3.7 Domain name system

The Validated Design assumes that you already have a public DNS hosted zone, in your cloud provider's DNS service (Route 53 for AWS, Azure DNS for Azure, Cloud DNS for GCP) or another

DNS service. This is a prerequisite for generating TLS certificates that secure traffic between Vault nodes.

If you intend to use a public cloud for DNS, the HVD module creates

- An A record pointing to the load balancer that it also created during deployment
- A failover record pointing to the same target location. You can specify the DNS hostname to the provided Terraform deployment modules, which create the necessary records in the public cloud DNS service.

3.8 Certificates

The provided Terraform deployment modules read these certificates and base64 encode them before uploading to the public cloud secrets management service for the life of the initial install process.

- `vault-public.pub` - the TLS certificate itself
- `vault-private.key` - the certificate's private key
- `ca.pub` - the certificate authority bundle file

3.9 IAM

The provided Terraform deployment modules require an IAM role with necessary permissions to deploy and manage all required AWS components. The module can create this role on your behalf or use an existing one that you create yourself. If you choose to create the IAM role yourself, ensure the role contains the following policies and permissions:

AWS

Service	Permissions	Resources
EC2 (for cloud auto-join)	DescribeInstances DescribeVolumes DescribeTags	All (*)
SSM*	(Attach `AmazonSSMManagedInstanceCore` managed policy)	n/a
KMS	ReEncrypt GenerateRandom GenerateDataKey* Encrypt DescribeKey Decrypt CreateGrant EnableKeyRotation	Vault KMS key
Autoscaling	CompleteLifecycleAction	Vault module ASG hook
Secrets Manager	GetSecretValue	Vault license Vault CA bundle Vault public cert Vault private key
CloudWatch	PutMetricData PutRetentionPolicy PutLogEvents DescribeLogStreams DescribeLogGroups CreateLogStreams CreateLogGroups	Vault log group
S3	PutObject ListBucket ListBucketVersions GetObject GetBucketLocation DeleteObject	Automated snapshots bucket

Azure

Service	Permissions	Resources
Virtual Machines (for cloud auto-join)	Microsoft.Compute/virtualMa Mi-	All (*)

4 Deploying Vault using Terraform

HashiCorp provides a set of official Terraform modules to make it easier to deploy a Vault Enterprise environment that adheres to the requirements and standards laid out in this HashiCorp Validated Design.

> Tip
>
> HashiCorp Professional Services or one of our trusted partners can help you accelerate your deployment of Vault Enterprise, using the provided modules. For more information, contact your HashiCorp account team.

4.1 Platform-specific guidance

Select your cloud provider below for further guidance.

AWS

HashiCorp provides an official HVD Module to deploy Vault Enterprise on AWS EC2.Before deployment, you will need to deploy the prerequisite infrastructure in AWS.1) A functional VPC with at least three public and private subnets. 2) A Vault license that has been uploaded to AWS Secrets Manager. 3) A TLS private key and certificate, valid for the fully qualified domain name you plan to use with Vault, that have been uploaded to AWS Secrets Manager. 4) The ARN of a symetric encryption key created in AWS KMS.

Azure

HashiCorp provides an official HVD Module to deploy Vault Enterprise on Azure VMs.Before deployment, you will need to deploy the prerequisite infrastructure in Azure.1) An Azure resource group. 2) An Azure Key Vault to store the prerequisite secret material. 3) A functional virtual network with private subnets for the Vault instances. 4) A key created in the Azure Key Vault to be used for automatically unsealing Vault. 5) A Vault license that has been uploaded as a secret in the Azure Key Vault. 6) A TLS private key and certificate, valid for the fully qualified domain name you plan to use with Vault, that have been uploaded as secrets to the prerequisite Key Vault.

GCP

HashiCorp provides an official HVD Module to deploy Vault Enterprise on GCP GCE.Before deployment, you will need to deploy the prerequisite infrastructure in GCP.1) A functional VPC with a public and private subnet. 2) A GCP KMS Key Ring to hold the key for automatically unsealing Vault. 3) A key created within the Key Ring to be used for automatically unsealing Vault. 4) A Vault license that has been uploaded as a secret in Google Secret Manager. 5) A TLS private key and certificate, valid for the fully qualified domain name you plan to use with Vault, that have been uploaded as secrets to Secret Manager.

Using this module, you will be able to use Terraform to deploy a complete, end-to-end Vault Enterprise deployment inside of your own cloud account.

Note

The following steps outline a manual method of installing Vault using this module. This method of installation is primarily designed for you to test the execution of the module and get to know the design. We strongly recommend adapting these steps into a more controlled deployment process, such as HCP Terraform, Terraform Enterprise, or your existing CI/CD pipeline infrastructure.

Warning

NEVER store Terraform state in a version control system or in another unprotected location. Terraform state contains sensitive information and must be protected. For more information, refer to the HashiCorp Terraform documentation.

While we have made efforts throughout this document to provide prescriptive best practices, we recognize that each organization has their own unique requirements and constraints when it comes to deploying infrastructure. Wherever possible, we have attempted to represent the different considerations you will need to make when deploying Vault in your cloud environment within the context of this Terraform module. The module contains additional capabilities that you may wish to review if the variables from this module do not suit your specific needs.

4.2 Deployment sequence overview

1. Create the certificate files.
2. Obtain the license file.
3. Download the Vault CLI.
4. Download the Terraform CLI.
5. Deploy your prerequisite resources.
6. Obtain the HashiCorp Validated Design Terraform module for deploying Vault Enterprise.
7. Configure your cloud credentials.
8. Initialize your Terraform workspace.
9. Input your variables, including the values from your prerequisite deployment, in the module for deployment.
10. Create a Terraform plan.
11. Apply the plan.
12. Validate the cluster was created and is reachable.

13. Initialize the Vault cluster.

4.3 Preparation

4.3.1 Create the certificate files

Create a standard X.509 certificate that will be installed on the Vault servers. Refer to your organization's process on creating a new certificate that matches the DNS record you intend to direct users to when accessing Vault.

A total of three files will be needed as follows.

1. The certificate (`vault-public.pub`).
2. The certificate's private key (`vault-private.key`).
3. The bundle file from the certificate authority used to vend the certificate (`ca.pub`).

Keep these files to hand as you will need them later in the installation process.

4.3.2 Obtain the Vault Enterprise license file

Obtain the Vault Enterprise license file from your HashiCorp account team. This file contains a license key unique to your environment. The file will be named something like `vault.hclic`.

Keep this file handy, as you will need it later in the installation process.

4.3.3 Download and Install the Vault CLI

> Note
>
> This guide was tested using Vault 1.17.1+ent.

- Download the appropriate package for your operating system from the HashiCorp Releases site.
- Unzip the package.
- Move the `vault` binary (`vault.exe` for Windows) to a directory in your system's `PATH`.

4.3.4 Download and install the Terraform CLI

> Note
>
> This guide was tested using Terraform 1.6.2.

- Download the appropriate package for your operating system from the HashiCorp Releases site.
- Unzip the package.
- Move the `terraform` binary (`terraform.exe` for Windows) to a directory in your system's PATH.

4.3.5 Download the Terraform module

For the purpose of an automated deployment, HashiCorp and HashiCorp partners provide private Terraform modules to customize and support your deployment.

Once you have downloaded the module, navigate to the `examples/default/` directory. Use this as the base working directory during the installation process.

4.3.6 Configure cloud credentials

Ensure the correct cloud credentials are in place and accessible to Terraform. The credentials you use must have sufficient permissions to perform the necessary actions that Terraform will be performing.

AWS

Terraform can read AWS credentials from:* Credentials file: typically located at `$HOME/.aws/credentials` (`%UserProfile%\.aws\credentials` on Windows). * Environment variables as follows. - `AWS_ACCESS_KEY_ID` - `AWS_SECRET_ACCESS_KEY` - `AWS_SESSION_TOKEN` (if using an IAM role or other expiring credentials) - `AWS_DEFAULT_REGION`For complete details on how to configure AWS credentials for Terraform, see the HashiCorp Terraform AWS provider documentation.

Azure

Configure Azure credentials by running `az login` with your assumed credentials. For complete details on how to configure Azure credentials for Terraform, see the HashiCorp Terraform Azure provider documentation.

GCP

Terraform can read GCP credentials from:* Credentials file: typically located at `$HOME/.config/gcloud/application_default_credentials.json` (`%APPDATA%\gcloud\application_default_credentials.json` on Windows). * Environment variables as follows. - `GOOGLE_CREDENTIALS`For complete details on how to configure GCP credentials for Terraform, see the HashiCorp Terraform GCP provider documentation.

4.4 Installation

4.4.1 Initialize Terraform

Run `terraform init` to initialize your Terraform workspace. Inspect the output to ensure that all providers and modules are successfully downloaded, and that there are no outstanding errors before continuing.

4.4.2 Configure variables for deployment

> **Warning**
>
> You can only configure variables for the installation module's `terraform.tfvars` file after all the prerequisite resources are available. You will need to supply values from the prerequisites to the Vault module.

Review the `terraform.tfvars.example` file HashiCorp maintains in the `examples/default/` directory for explanations of each relevant variable. There is a `terraform.tfvars.example` file in the respective module for each public cloud provider. Copy this file to a file called `terraform.tfvars`, and then fill in the values for each declared variable with the applicable values for your environment.

4.4.3 Create and apply Terraform plan

From the `examples/default/` directory, generate a Terraform plan with the following command:

```
terraform plan -out plan.out
```

Review the plan output to see the changes that will be applied, then apply the changes with this command:

```
terraform apply plan
```

Confirm the changes by typing `yes` when prompted.

4.4.4 Validate installation

Run the following command:

```
terraform output
```

Using the terraform output that references the load balancer name or ip address, create a new DNS entry that matches your TLS certificate and points to the load balancer for the Vault cluster. Set the following environment variables:

```
$ export VAULT_ADDR="https://vault.example.com:8200/"
```

Test that the Vault cluster is running and reachable by running:

```
vault status
```

4.5 Initialize

4.5.1 Create GPG public keys

For each individual who will be receiving a key share, create their own unique GPG public key:

```
$ cat > alice_key.conf << EOF
%echo Generating a basic OpenPGP key for Alice
Key-Type: 1
Key-Length: 4096
Subkey-Type: 1
Subkey-Length: 4096
Name-Real: Alice
Name-Comment: Alice is a Vault PGP user
Name-Email: alice@example.com
Expire-Date: 1
Passphrase: recede-yard-unwilling-shrouded
%commit
%echo done
EOF


$ gpg --full-gen-key --batch alice_key.conf…

$ gpg --output alice_key.pub --export alice@example.com
```

4.5.2 Initialize Vault

Pass in the GPG public key from each participating individual to initialize Vault with the API, CLI, or UI. Once Vault initializes, it returns the key shares and the initial root token, each encrypted with the GPG public keys you passed in at initialization time.

```
$ vault operator init \
  -recovery-pgp-keys “alice_key.pub,bob_key.pub,carol_key.pub,dan_key.pub,frank_key.
     pub" \
  -root-token-pgp-key "alice_key.pub"


Recovery Key 1: wcFMA7Np...JCbg=
Recovery Key 2: wcFMAw4D...5LY4=
Recovery Key 3: wcFMA8d6...G8r0=
Recovery Key 4: wcFMA4ly...6ELY=
Recovery Key 5: wcFMA0eb...3hRg=


Initial Root Token: …wcFMA79K9MhW9


Success! Vault is initialized


Recovery key initialized with 5 key shares and a key threshold of 3. Please
securely distribute the key shares printed above.
```

Record the `root_token` from the output. The token is encrypted and base64-encoded. The root token will allow you to authenticate with Vault and begin the configuration of user authentication.

Each individual should now record their unique recovery key and store it safely. This is their key share, which is now encrypted and base64-encoded.

This concludes the key ceremony.

You should now have a running cluster. Decrypt the root token and log into Vault:

```
$ VAULT_ADDR=$(terraform output vault_addr) VAULT_TOKEN="<hvs-TOKEN-value-from-key-
    ceremony>" vault login
```

5 Deploying Vault Enterprise in a private datacenter

5.1 Overview

This section details the steps to create a Vault cluster manually in a private datacenter. This guide assumes that you have already read the Detailed Design section of this guide and have a basic understanding of the Vault architecture and the steps required to deploy Vault in a private datacenter.

Throughout this section we will use the following names and IP addresses for the Vault servers:

DNS Name	IP Address	Node Type	Location
vault-1.domain	10.0.0.1	voter	zone1
vault-2.domain	10.0.0.2	voter	zone2
vault-3.domain	10.0.0.3	voter	zone3
vault-4.domain	10.0.0.4	non-voter	zone1
vault-5.domain	10.0.0.5	non-voter	zone2
vault-6.domain	10.0.0.6	non-voter	zone3
vault.domain	10.0.0.100	(load balancer)	(all zones)

These names and IP addresses are used for illustrative purposes only. Use the names and IP addresses that are appropriate for your environment.

5.2 Prepare

Create or obtain the files that you will later deploy onto the Vault servers. The Vault binaries are pulled via `curl` command during the Installation phase.

5.2.1 TLS

Create an X.509 certificate that will be installed onto each of the Vault servers. Refer to your organization's process on creating a new certificate that matches the DNS record you intend to direct users to when accessing Vault - in this case, the DNS record pointing at the load balancer: `vault.domain`.

Three files will be needed:

1. The certificate (`vault-public.pub`).
2. The certificate's private key (`vault-private.key`).
3. The bundle file from the certificate authority used to vend the certificate (`ca.pub`).

Note

Only one set of these files is required. These files will be copied onto all nodes.

5.2.2 License

Obtain your active Vault Enterprise License file. If you do not have this file, contact your HashiCorp account team.

5.2.3 Server configs

Using what you have learned from previous sections of this guide, create 6 configuration files, one for each server. The configuration for each server will be slightly different, depending on which redundancy zone it lives in, and whether it is a voting or a non-voting node.

Voting node example file

```
# config.hcl for vault1 (voter node example)
api_addr = "https://10.0.0.1:8200"
cluster_addr = "https://10.0.0.1:8201"
ui = true
disable_mlock = true

license_path = "/etc/vault.d/vault.hclic"

listener "tcp" {
  address     = "10.0.0.1:8200"
  cluster_address = ""10.0.0.1:8201
  tls_client_ca_file = "/etc/vault.d/ca.pub"
  tls_cert_file = "/etc/vault.d/vault-public.pub"
  tls_key_file  = "/etc/vault.d/vault-private.key"
}

storage "raft" {
  path    = "/var/lib/vault/data"
  node_id = "vault1"
  autopilot_redundancy_zone = "zone1"

  retry_join {
    leader_api_addr     = "https://10.0.0.1:8200"
    leader_ca_cert_file = "/etc/vault.d/ca.pub"
    leader_client_cert_file = "/etc/vault.d/vault-public.pub"
    leader_client_key_file  = "/etc/vault.d/vault-private.key"
    leader_tls_servername = "vault."domain
}

  retry_join {
    leader_api_addr     = "https://10.0.0.2:8200"
    leader_ca_cert_file = "/etc/vault.d/ca.pub"
    leader_client_cert_file = "/etc/vault.d/vault-public.pub"
    leader_client_key_file  = "/etc/vault.d/vault-private.key"
    leader_tls_servername = "vault."domain
  }

  retry_join {
    leader_api_addr     = "https://10.0.0.3:8200"
    leader_ca_cert_file = "/etc/vault.d/ca.pub"
    leader_client_cert_file = "/etc/vault.d/vault-public.pub"
    leader_client_key_file  = "/etc/vault.d/vault-private.key"
    leader_tls_servername = "vault."domain
  }

  retry_join {
    leader_api_addr     = "https://10.0.0.4:8200"
    leader_ca_cert_file = "/etc/vault.d/ca.pub"
    leader_client_cert_file = "/etc/vault.d/vault-public.pub"
    leader_client_key_file  = "/etc/vault.d/vault-private.key"
    leader_tls_servername = "vault."domain
  }

  retry_join {
    leader_api_addr     = "https://10.0.0.5:8200"
    leader_ca_cert_file = "/etc/vault.d/ca.pub"
    leader_client_cert_file = "/etc/vault.d/vault-public.pub"
```

Non-voting node example file

```
# config.hcl for vault4 (non-voter node example)

api_addr = "http://10.0.0.4:8200"
cluster_addr = "https://10.0.0.4:8201"
ui = true
disable_mlock = true

license_path = "/etc/vault.d/vault.hclic"

storage "raft" {
  path    = "/var/lib/vault/data"
  node_id = "vault4"
  autopilot_redundancy_zone = "zone1"
}

listener "tcp" {
  address     = "10.0.0.4:8200"
  cluster_address = ""10.0.0.4:8201
  tls_client_ca_file = "/etc/vault.d/ca.pub"
  tls_cert_file = "/etc/vault.d/vault-public.pub"
  tls_key_file  = "/etc/vault.d/vault-private.key"

  retry_join_as_non_voter = true

  retry_join {
    leader_api_addr     = "https://10.0.0.1:8200"
    leader_ca_cert_file = "/etc/vault.d/ca.pub"
    leader_client_cert_file = "/etc/vault.d/vault-public.pub"
    leader_client_key_file  = "/etc/vault.d/vault-private.key"
    leader_tls_servername = "vault."domain
  }

  retry_join {
    leader_api_addr     = "https://10.0.0.2:8200"
    leader_ca_cert_file = "/etc/vault.d/ca.pub"
    leader_client_cert_file = "/etc/vault.d/vault-public.pub"
    leader_client_key_file  = "/etc/vault.d/vault-private.key"
    leader_tls_servername = "vault."domain
  }

  retry_join {
    leader_api_addr     = "https://10.0.0.3:8200"
    leader_ca_cert_file = "/etc/vault.d/ca.pub"
    leader_client_cert_file = "/etc/vault.d/vault-public.pub"
    leader_client_key_file  = "/etc/vault.d/vault-private.key"
    leader_tls_servername = "vault."domain
  }

  retry_join {
    leader_api_addr     = "https://10.0.0.5:8200"
    leader_ca_cert_file = "/etc/vault.d/ca.pub"
    leader_client_cert_file = "/etc/vault.d/vault-public.pub"
    leader_client_key_file  = "/etc/vault.d/vault-private.key"
    leader_tls_servername = "vault."domain
  }

  retry_join {
```

Set aside these files for later in the installation process.

5.3 Build

5.3.1 Servers

1. Identify the availability zones within your datacenter where your Vault servers will live. For the rest of this document, we will refer to these as `zone1`, `zone2`, etc.
2. Build the servers. Ensure there are 2 servers per each of the 3 redundancy zones, for a total of 6 servers. For the rest of this document, we will refer to these servers as `vault1`, `vault2`, etc.
3. Ensure you can log in to each server as the root user (or a user with sudo privileges) via SSH or equivalent.

5.3.2 Load balancer

Configure your load balancer with a new frontend for Vault that will route incoming traffic on TCP port 8200 to the Vault servers on TCP port 8200. Refer to the Detailed Design section of the guide for more information on how to configure the load balancer.

5.3.3 Snapshot storage

Ensure that your snapshot storage location is created and that the Vault service can successfully write to it. If you will be using a remote network filesystem, ensure that it is mounted at the correct location on all Vault servers.

Configuration of Vault snapshots can be found in Vault: validated design guidance.

5.4 Install Vault

Perform these steps on all Vault servers:

1. Create a `vault` user. Create directories for Vault configuration owned by this user.

```
$ useradd --system --user-group --shell /bin/false vault
$ mkdir -p /etc/vault /var/lib/vault
$ chown -R vault:vault /etc/vault /var/lib/vault /usr/local/bin/vault
```

3. Download the Vault Enterprise package from Hashicorp. Unzip the package and move the `vault` binary to a shared `PATH` location such as `/usr/local/bin`, owned by the `vault` user.

```
$ curl -O https://releases.hashicorp.com/vault/1.15.0+ent/vault_1.15.0+
    ent_linux_amd64.zip
$ unzip vault_1.15.0+ent_linux_amd64.zip
$ mv vault /usr/local/bin/
$ chown vault:vault /usr/local/bin/vault
```

> **Note**
>
> As of this writing, the current version of Vault Enterprise is 1.17.1+ent. Update the curl and unzip commands to reflect the desired version of Vault Enterprise.

4. Copy the files created in the prepare section above (license file, certificate files, and relevant config file) to this server at `/etc/vault`. Ensure that the files are owned by the `vault` user and that the permissions are set to `640`.

```
$ chown vault:vault /etc/vault.d/*
$ chmod 640 /etc/vault.d/*
$ ls -l /etc/vault.d/

-rw-r---- 1 vault vault 0 Sep 22 12:45 ca.pub
-rw-r---- 1 vault vault 0 Sep 22 12:45 config.hcl
-rw-r---- 1 vault vault 0 Sep 22 12:45 vault.hclic
-rw-r---- 1 vault vault 0 Sep 22 12:45 vault-public.pub
-rw-r---- 1 vault vault 0 Sep 22 12:45 vault-private.key
```

5. Create a systemd unit file for the Vault service, then load it into systemd. Note that the `ExecStart` line runs the `vault` binary pointing to your `config.hcl` file:

```
$ cat << EOF >> /etc/systemd/system/vault.service
[Unit]
Description="HashiCorp Vault - A tool for managing secrets"
Documentation=https://developer.hashicorp.com/vault/docs
Requires=network-online.target
After=network-online.target
ConditionFileNotEmpty=/etc/vault.d/config.hcl
StartLimitIntervalSec=60
StartLimitBurst=3

[Service]
Type=notify
EnvironmentFile=/etc/vault.d/vault.env
User=vault
Group=vault
ProtectSystem=full
ProtectHome=read-only
PrivateTmp=yes
PrivateDevices=yes
SecureBits=keep-caps
AmbientCapabilities=CAP_IPC_LOCK
CapabilityBoundingSet=CAP_SYSLOG CAP_IPC_LOCK
NoNewPrivileges=yes
ExecStart=/usr/local/bin/vault server -config=/etc/vault.d/config.hcl
ExecReload=/bin/kill --signal HUP $MAINPID
KillMode=process
KillSignal=SIGINT
Restart=on-failure
RestartSec=5
TimeoutStopSec=30
LimitNOFILE=65536
LimitMEMLOCK=infinity
LimitCORE=0

[Install]
WantedBy=multi-user.target
EOF


$ systemctl daemon-reload
$ systemctl enable vault
```

6. Start the Vault service.

```
$ systemctl start vault
```

5.4.1 Initialize

Create GPG public keys

For each individual who will be receiving a key share, create their own unique GPG public key:

```
$ cat > alice_key.conf << EOF
%echo Generating a basic OpenPGP key for Alice
Key-Type: 1
Key-Length: 4096
Subkey-Type: 1
Subkey-Length: 4096
Name-Real: Alice
Name-Comment: Alice is a Vault PGP user
Name-Email: alice@example.com
Expire-Date: 1
Passphrase: recede-yard-unwilling-shrouded
%commit
%echo done
EOF

$ gpg --full-gen-key --batch alice_key.conf
...
$ gpg --output alice_key.pub --export alice@example.com
```

5.4.2 Initialize Vault

> **Warning**
>
> The following section should be performed on only one of the nodes. Failure to follow this could result in multiple single-node clusters, rather than a single cluster containing all nodes.

Pass in the GPG public key from each participating individual to initialize Vault with the API, CLI, or UI. Once Vault initializes, it returns the key shares, each one encrypted with the GPG public keys which you passed in at initialization time.

```
$ vault operator init \
  -pgp-keys “alice_key.pub,bob_key.pub,carol_key.pub,dan_key.pub,frank_key.pub" \
  -root-token-pgp-key "alice_key.pub"


Unseal Key 1: wcFMA7Np...JCbg=
Unseal Key 2: wcFMAw4D...5LY4=
Unseal Key 3: wcFMA8d6...G8r0=
Unseal Key 4: wcFMA4ly5...6ELY=
Unseal Key 5: wcFMA0eb...3hRg=


Initial Root Token: …wcFMA7Np2mGO


Vault initialized with 5 key shares and a key threshold of 3. Please securely
distribute the key shares printed above. When the Vault is re-sealed,
restarted, or stopped, you must supply at least 3 of these keys to unseal it
before it can start servicing requests.


Vault does not store the generated root key (previously known as master key).
Without at least 3 keys to reconstruct the root key, Vault will remain permanently
    sealed!


It is possible to generate new unseal keys, provided you have a quorum of existing
    unseal keys shares. See "vault operator rekey" for more information.
```

2. *Record the root token value from this output. You will need it in order to log into Vault initially.*
3. Each individual should now record their unique unseal key and store it safely. *This is their key share*, which is now encrypted and base64-encoded.

Unseal Vault

To unseal Vault using encrypted key shares, each individual will need to both base64-decode and decrypt their unseal key before sending it to the Vault server. When decrypting with GPG each individual will also need to pass the unique passphrase that was supplied during the creation of their own GPG key:

```
$ cat encoded-encrypted-keyshare1 | base64 --decode > encrypted-keyshare1
$ gpg --passphrase recede-yard-unwilling-shrouded --decrypt encrypted-keyshare1 >
    keyshare1
$ vault operator unseal keyshare1


gpg: encrypted with 4096-bit RSA key, ID 91002A8F909C6714, created 2023-04-17
      "Alice (Alice is a Vault PGP user) <alice@example.com>"
Key                Value
---                -----
Seal Type          shamir
Initialized        true
Sealed             true
Total Shares       5
Threshold          3
Unseal Progress    1/3…
```

Repeat this process until a sufficient number of keyshares have been supplied to Vault. At this point the Vault server is now unsealed. The output of the final unseal command should look something like the below.

```
gpg: encrypted with 4096-bit RSA key, ID 6ED85F446B6FAA27, created 2023-04-17
      "Dan (Dan is a Vault PGP user) <dan@example.com>"
Key                    Value
---                    -----
Seal Type              shamir
Initialized            true
Sealed                 false
Total Shares           5
Threshold              3…
```

This concludes the key ceremony.

5.4.3 Form the cluster

You should now have a functional, initialized and unsealed cluster, which contains one active node: `vault1`. You can confirm this by setting the VAULT_TOKEN environment variable to the root token generated during the init step and then running:

```
$ vault operator raft list-peers
```

You should now see that the cluster is comprised of six nodes: one active node, and five standby

nodes. To view the voter/non-voter status of each node, you can run:

```
$ vault operator raft autopilot state
```

Congratulations, you now have a running cluster! You can now test that you are able to log into Vault:

```
$ export VAULT_TOKEN=<root_token_value>
$ vault login -no-store $VAULT_TOKEN
```

6 Post-installation configuration

> Note
>
> This page covers first-run configuration after the Vault cluster is deployed. Ongoing namespace, authentication, and policy administration is covered in the Administration Guide.

This section describes how the product will be configured after installation (in case of self-hosted) and how it will be configured after the initial admin account is created.

6.1 Overview

Before you start populating Vault with secrets, there are initial configuration tasks that you should complete. This document covers configuration for both HCP Vault and Vault Enterprise and we make a note if any of the configuration items do not apply to a certain product type.

Tasks in this section are below.

- The process of configuring the audit logs - For Vault Enterprise only.
- Namespace design and recommended structure.

6.1.1 Prerequisites

- You have reviewed and implemented the HashiCorp Validated Document (HVD) for Vault Solution Design.
- You have a running Vault cluster that is initialized and unsealed.
- You have a valid root token for your Vault cluster.

6.2 Configure audit logs

> Note
>
> You do not need to configure audit logs for HCP Vault. Audit logs are accessible to production tier clusters, they are stored in an encrypted Amazon S3 bucket in the same region as the cluster. HCP Vault supports streaming audit logs to a variety of destinations.

Audit logs are critical for Vault administrators to ensure proper usage, access, and compliance with established security policy. In this section, you will learn about the different types of audit devices and how to enable audit logging in your Vault cluster.

Each line in the audit log is a JSON object containing all of the information for any given request and corresponding response. By default, sensitive information is hashed before it is logged. Audit logs can be used by administrators to monitor the health of the service and to troubleshoot issues, or by compliance auditors to ensure secrets are being accessed and used securely. Each audit log entry contains data such as client IP address, the time of the request, the requested action, and the resulting data from Vault.

> Note
>
> Audit device logs are separate and unrelated to Vault operational logs. Operational logs are typically gathered by the operating system journal from standard output and standard error while Vault is running, and hold a different set of information.

When you enable an audit device in Vault, most strings contained within requests and responses are hashed with a salt using HMAC-SHA256. The purpose of the hash is so that secrets are not in plaintext within your audit logs. However, you are still able to check the value of secrets by generating HMACs yourself; this can be done with the audit device's hash function and salt by using the `/sys/audit-hash` API endpoint.

Audit logs are enabled in Vault by configuring an audit device. Audit devices define the destination for audit log data. There are three types of audit devices: file, syslog, and socket.

6.2.1 Types of audit devices

File audit device

The file audit device writes logs to a file. New logs are appended to the log file. The device does not support log rotation. It is up to the operator to use third-party tools such as *logrotate* to manage log rotation. Sending a SIGHUP to the Vault process will cause file audit devices to close and re-open their underlying file, which can assist with log rotation needs.

> Warning
>
> It is important to rotate and archive audit log files to prevent it from growing to a size that consumes the entire disk space. Vault will not respond to any API requests if there is a blocked file audit device.

Configuration details for this option can be found here.

Log file rotation

`logrotate` is a common Linux system utility designed to ease administration of systems that generate large numbers of log files. It allows automatic rotation, compression, removal, and mailing of log files. Each log file may be handled daily, weekly, monthly, or when it grows too large. `logrotate` is available on many Linux distributions, although the default configuration may vary between distributions. Normally, it is run as a daily `cron` job. It will not modify a log more than once in one day unless the criterion for that log is based on the log's size and `logrotate` is being run more than once each day, or unless the `-f` or `--force` option is used.

Below is an example of `logrotate` configuration (adjust the retention and path to systemctl for your environment):

```
/opt/vault/log/vault_audit.log {
  daily
  rotate 7
  notifempty
  missingok
  compress
  delaycompress
  postrotate
    # systemd unit file should be set to send SIGHUP to Vault process on reload, i.e.
        ExecReload=/bin/kill --signal HUP $MAINPID
    /bin/systemctl reload vault 2> /dev/null || true
  endscript
  create 0644 vault vault
}
```

Since the audit log is verbose, we recommend that you only keep a few days of audit logs locally and export old logs to archive storage.

Syslog audit device

The `syslog` audit device writes audit logs to `syslog`. It does not support remote `syslog` destinations and always sends audit logs to a local `syslog` agent. The `syslog` audit device is usually configured to write to a local file for either log collection or as a second option for audit device configuration.

Configuration details for this option can be found here

Socket audit device

The `socket` audit device writes to a TCP, UDP, or UNIX socket. Due to the unreliable nature of the underlying protocol, we do not recommend enabling the `socket` audit device unless it is absolutely necessary. If you do enable it, always enable a secondary non-socket audit device to ensure accuracy and to guarantee that audit logs will not be lost.

Configuration details for this option can be found here.

6.2.2 Multiple audit devices

An otherwise-successful request will fail if it cannot be logged to at least one configured audit device. Failure to log to at least one audit device will prevent Vault from servicing requests (see blocked audit device). This is by design to ensure that all requests and responses are captured correctly.

We strongly recommend that you enable at least two audit devices of different types for two reasons:

Improved Availability

There are two types of audit device failures: blocking and non-blocking.

- A blocking failure is one where an attempt to write to the audit device stalls without returning an error. This is unlikely with a local disk device, but could occur with a network-based audit device.

- A non-blocking failure is one where an attempt to write to the audit device returns an error and no audit log is written.

When multiple audit devices are enabled, if any of them fail in a non-blocking fashion, Vault requests can still complete successfully provided at least one audit device successfully writes the audit record. If any of the audit devices fail in a blocking fashion however, Vault requests will hang until the blocking is resolved.

Checking and verification

Configuring multiple audit devices provides you not only with redundant copies, but also a way to check for data tampering in the logs themselves. We recommend that you set up one audit log for analysis, and another for secure storage and archiving. In case there are concerns about the integrity of the analysis logs, you can refer to the archived logs for verification. For archival purposes, a second audit device should be enabled to write to a filesystem or syslog destination which is configured with strict access control permissions. Read-only access to these logs can be granted in the case where there is a need to reconcile with the main audit log, but otherwise these logs can remain untouched. This ensures there is an unaltered version of the audit log for security review. When writing to a `file` audit backend it is important to monitor disk space on the disk where the logs are being written. If disk space fills up, it will result in a blocked audit device, preventing Vault from responding to requests.

6.2.3 Enabling an audit device

When a Vault server is first initialized, no auditing is enabled. Audit devices must be enabled by a root user using the CLI, API, or Terraform.

Note

Audit device configuration is replicated to all nodes within a cluster by default, and to performance/DR secondaries for Vault Enterprise clusters. Each node in the cluster writes to its own audit log, in the same locations as the active node. Before enabling an audit device, ensure that all nodes within the cluster(s), including your DR and performance secondary clusters, will be able to successfully log to the audit device to avoid Vault being blocked from serving requests. Audit logs from all nodes in a Vault cluster need to be analyzed to audit any event. Thus it is best practice to use a centralized logging solution. An audit device can also be limited to only the nodes within the cluster using the local parameter. This is useful if you want to have different audit device configurations on replicated clusters.

We recommend that you enable a `file` audit device as well as a `syslog` audit device (see Multiple Audit Devices).

Step 1: Ensure you have the correct system permissions

It is important to ensure that the Vault process user has the correct system permissions to write to the configured audit device.

For the `file` audit device, the Vault process user must have write access to the location of the file. For the `syslog` audit device, the Vault process user must have the correct capabilities such as `CAP_SYSLOG` and permissions where required to write to the system log.

Step 2: Enable the audit device

First, set your root token using the Vault CLI.

```
$ export VAULT_ADDR=https://<vault FQDN>:8200
$ export VAULT_TOKEN=<your root token>
```

Note

The `VAULT_TOKEN` environment variable sets the authentication token that the Vault CLI will use for all subsequent requests. The `VAULT_ADDR` environment variable tells the Vault CLI where to send all subsequent requests. See our documentation for a list of valid environment variables.

The following command enables a `file` audit device. The output logs are stored in the `/vault/vault-audit.log` file.

```
$ vault audit enable file file_path=/vault/vault-audit.log
```

If the Vault process user does not have permission to write to the file provided in the `file_path` parameter, you can observe the error below.

```
Error enabling audit device: Error making API request.

URL: PUT http://localhost:8200/v1/sys/audit/file
Code: 400. Errors:

* sanity check failed; unable to open "/vault/vault-audit.log" for writing: open /
    vault/vault-audit.log: permission denied
```

The following command enables a `syslog` audit device, specifying the syslog facility and tag to use.

```
$ vault audit enable syslog tag="vault" facility="AUTH"
```

If syslog is not accessible on the system, you can observe errors in the operational log like this when Vault tries to write to it, and when you first try to enable it.

```
[ERROR] enable audit mount failed: path=syslog/ error="Unix syslog delivery error"
[ERROR] core: failed to audit response: request_path=sys/audit/syslog error=1 error
    occurred:
* no audit backend succeeded in logging the response
```

The error `Unix syslog delivery error` can mean that the syslog service is not enabled on the host or that Vault is not able to access it. This can often be due to restrictions imposed by SELinux configuration on the host, for example. In order to check whether SELinux is actively prohibiting access to a resource, the operating mode can temporarily be changed to permissive using the `setenforce` utility. A more permanent solution would include enabling SELinux debugging and using packages such as `setools` and `settroubleshoot` to obtain information about specific operation denials.

Warning

Audit messages generated for some operations can be quite large, and can be larger than a maximum-size single UDP packet. Because UDP is a connectionless protocol, if a log message is larger than the maximum size UDP packet, then that audit log message will fail silently. Vault will have no knowledge that the message was too large. If possible with your syslog daemon, configure a TCP listener. Because TCP protocol is "connection-oriented" Vault will have awareness whether syslog messages have been successfully received. This can result in a blocked audit device if the TCP connections are unsuccessful. To avoid this possibility, consider using a `file` backend and having syslog configured to read entries from the file.

Note

A typical audit log entry can be 1kb-3kb, meaning a node servicing 10,000 requests an hour can write 10-30mb of data. We recommend using a log rotation solution such as `logrotate` or equivalent to avoid the local file system logs from becoming full, and transfer the logs to external storage in the event the node loses a disk or is compromised. We also recommend that you configure the audit logs to write to a separate logical volume to avoid any disk IO contention with Vault's internal storage when using integrated storage.

For more information on how to use the audit log, please refer to the Audit Usage section of this document.

6.3 Namespaces

After configuring the audit log, the next step is to think about how to organize the data within Vault. Many organizations opt to implement Vault as a service, where a central platform team is responsible for the day-to-day operations of Vault, while development teams simply utilize Vault's capabilities. To enable this model, you need a mechanism to isolate groups of resources within a single cluster, and this is where Vault namespaces fit in. Read, and map your organization's plans for multi-tenancy in Vault to the Vault Namespace and Mount Structuring Guide.

6.3.1 What are Vault namespaces

Namespaces are a method by which a single Vault cluster can be divided into multiple sub-clusters and managed individually. Each namespace can be assigned different login paths and support creating and managing data isolated to their namespace.

6.3.2 When to use Vault namespaces

Namespaces are designed to address two use cases.

Tenant isolation

There is a need for isolation between the users in policies, secrets, and identities, typically as a result of compliance regulations such as GDPR or internal security policy.

Self-management

There is a need to provide delegated administrative privileges for a team to author their own policies and manage their own namespace. For example, if you have a development team who is comfortable with Vault and wishes to self-manage and operate in their own namespace.

Namespace antipatterns

Vault namespaces are subject to limits and maximums within Vault's backend storage. The effective storage limit on the number of Vault namespaces is a result of the fact that each namespace must have at least two secret engine mounts (for sys and identity), one local secret engine (cubbyhole) and one auth engine mount (token). Depending on your organizations namespace structure (ie. how many auth engines and secret engines you mount under each namespace) the effective storage maximum will vary.

Additionally, administering a large number of namespaces can become difficult and any restructuring can be problematic. There are a few anti-patterns that should be avoided when planning Vault Enterprise namespaces:

- Not clearly defining criteria for a namespace: If namespaces are created ad-hoc without a clear plan or criteria, the namespace structure in an enterprise deployment will quickly become difficult to manage.
- Strong misalignment to organizational structure: Vault namespaces should ideally be roughly aligned to the level of granularity across lines of businesses (LOBs), divisions, teams, services, apps that needs to be reflected in Vault's end-state design. If the namespace layout bears

no resemblance to the level of self-service required in the organization, management can become difficult.

- Non-scalable model or over-segmentation: If an organization plans on provisioning a namespace for each application, but the organization has 10,000 applications to onboard, then Vault's storage limits will be hit and management of a large number of namespaces will be difficult.

Namespace performance impacts

In addition to the storage limits listed above, large numbers of namespaces can also have performance impacts on specific Vault workflows. For example, if the number of leases is also growing linearly or exponentially with the number of namespaces and mounts, then API requests might become slower if the growth is unbounded.

Additionally, a large list of namespaces can impact the time to complete a leader election. In order for a node to become an active leader in a Vault cluster, it has to load up all the namespaces and mounts. A large number of namespaces will potentially impact the amount of time it takes to complete this process. This document should be referred to should your organization be planning on creating a large number of namespaces mapped to individual app teams where large numbers of the same secret engine mounts will be used.

6.3.3 Useful resources

- Vault Enterprise namespaces
- Secure multi-tenancy with namespaces

7 Disaster recovery initial setup

> **Note**
>
> This page covers initial disaster recovery replication setup. For recurring failover, promotion, and failback operations, see the Administration Guide: Disaster recovery operations.

7.1 Overview

Prior to configuring Vault, it is essential to plan for potential infrastructure failures. HashiCorp advises that Vault Enterprise customers implement, at a minimum, a disaster recovery (DR) solution for their production clusters.

> Note
>
> HCP Vault cluster DR posture is managed by the HCP Platform. For more information please review the High Availability and Disaster Recovery documentation.

7.1.1 Prerequisites

- You have reviewed and implemented the HashiCorp Vault Installation Guide.
- You have deployed two separate Vault clusters, one acting as the DR primary, and the other acting as at the DR secondary.
- The DR Vault clusters are initialized and unsealed.
- You have a valid root tokens for your clusters.

7.2 Types of cluster replication in Vault Enterprise

Vault can be extended across data centers or cloud regions using the replication feature of Vault Enterprise. Vault replication operates on a leader/follower model, wherein a leader cluster (known as a *primary*) is linked to a series of follower secondary clusters to which the primary cluster replicates data.

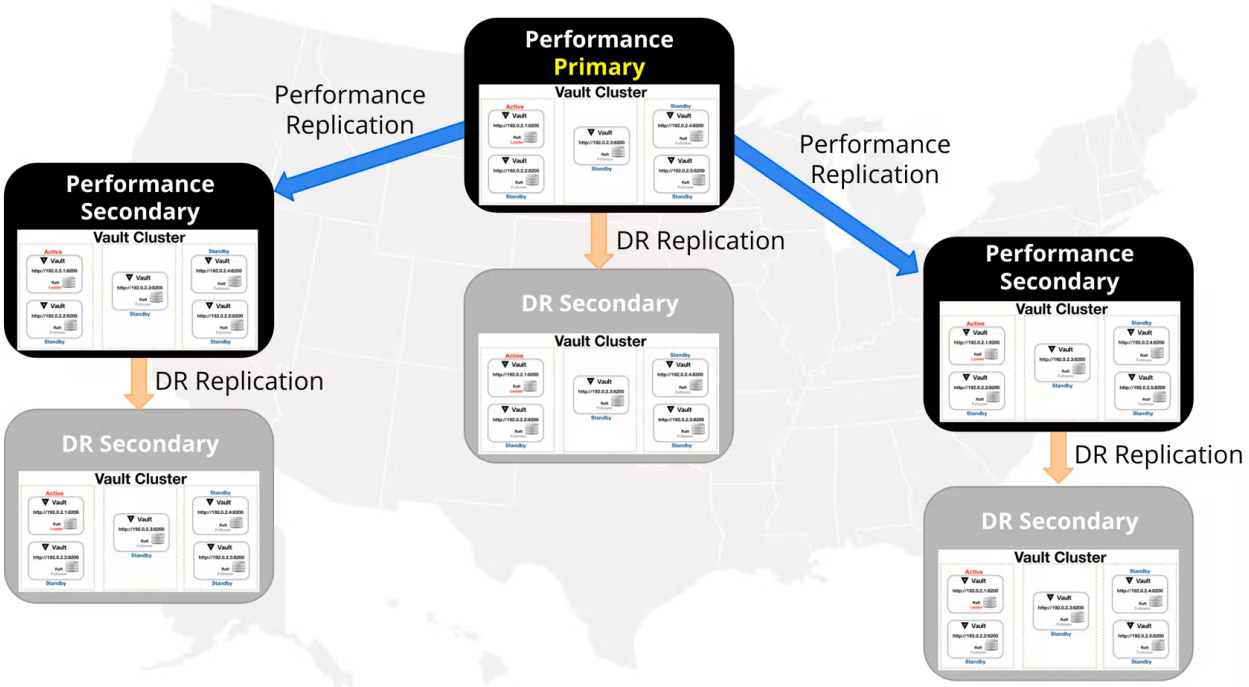


Figure 1: Vault DR Architecture

The data replicated between the primary and secondary cluster is determined by the type of replication: disaster recovery (DR) or performance replication (PR).

A *DR secondary cluster* is a complete mirror of its primary cluster. DR Replication is designed to be a mechanism to protect against catastrophic failure of entire clusters. DR secondaries do not service read or write requests until they are promoted and become a new primary. A DR secondary cluster acts as a warm standby cluster.

A *PR secondary cluster* replicates everything from the primary *except* for tokens, leases (dynamic secrets), and any local secrets engines, and is covered in this document.

7.3 Setting up DR replication

At this point, you should have two Vault Enterprise clusters: one that will act as the **DR primary cluster**, and another that will become the **DR secondary cluster**. We strongly advise carefully reading and implementing the details provided in the official documentation on disaster recovery replication set up.

7.3.1 Configure DR operation token

A DR operation token is required to promote a DR secondary cluster as the new primary. This token should be created in advance and stored safely outside of Vault in case of a failure condition that would warrant the promotion of the DR secondary cluster.

Because the DR operation token has the ability to fundamentally modify your Vault clusters' topology and operation, it should be securely stored and time limited with an expiration date. While the public document regarding DR operation token strategy suggests that you can create a batch token as an alternative to a DR operation token, we recommend that you use the former, which can be generated from the DR primary cluster without your recovery keys (or unseal key if you are not using auto-unseal). The batch DR operation token has a fixed TTL and the Vault server will automatically delete it after it expires.

In addition to generating the DR operation token, automate its renewal and ensure that you store it outside of Vault's operational blast radius. This ensures that, in the event Vault becomes unavailable or inaccessible, you can still retrieve the token to perform a DR promotion.

The storage location must be external to Vault and resilient to the same failure scenarios. Suitable options include an object storage service (such as an S3 bucket), a shared password manager, or an on-premises filesystem. Regardless of the mechanism chosen, you must ensure that the token remains both accessible and valid for use during a failure condition.

> Note
>
> You must have permission to call the auth/token/create endpoint in the root namespace for this CLI command. In the Configure Admin Policy section, you will configure a policy with super administrators access to call this API in the root namespace.

The recommendation to create a *batch* DR operation token means the operations team have such but that this is valid for eight hours. Use the token in such a way that when a Vault operator comes on a shift, the operator generates a batch DR operation token with a TTL equal to the duration of the shift.

8 Performance replication architecture and setup

> **Note**
>
> This page covers performance replication architecture and initial setup. For recurring monitoring, see the Administration Guide: Performance replication operations.

8.1 Overview

As outlined in the Vault: validated design guidance, Vault Enterprise supports two types of cluster replication, Disaster Recovery (DR) Replication and Performance Replication (PR).

Vault enterprise performance replication allows for secret management and data encryption across regions, clouds, and data centers. Secrets, authentication methods, authorization policies, encryption keys, and other configuration data is replicated to be active and available in multiple locations in realtime.

A PR secondary cluster replicates everything from the primary except for tokens and leases (for dynamic secrets). Path filters and local mounts can also be used to control which secrets are moved across clusters. A PR secondary cluster can actively serve Vault client requests at the same time as a PR primary cluster. PR replication should primarily be implemented for two use cases:

- Horizontal scaling of "local" writes (token creation and dynamic secrets)
- Reducing latency by serving Vault traffic closer to where clients are deployed

8.2 Architecture

Both Vault disaster recovery and performance replication operate on a leader/follower model, wherein a leader cluster (known as a DR or PR primary) is linked to one or more follower (secondary) clusters to which the primary cluster replicates data via a write ahead log (WAL) stream. A given Vault cluster may serve simultaneously in the role of DR primary and PR primary. A cluster can also simultaneously serve as a DR primary and PR secondary. A cluster cannot serve as a DR secondary and PR secondary.

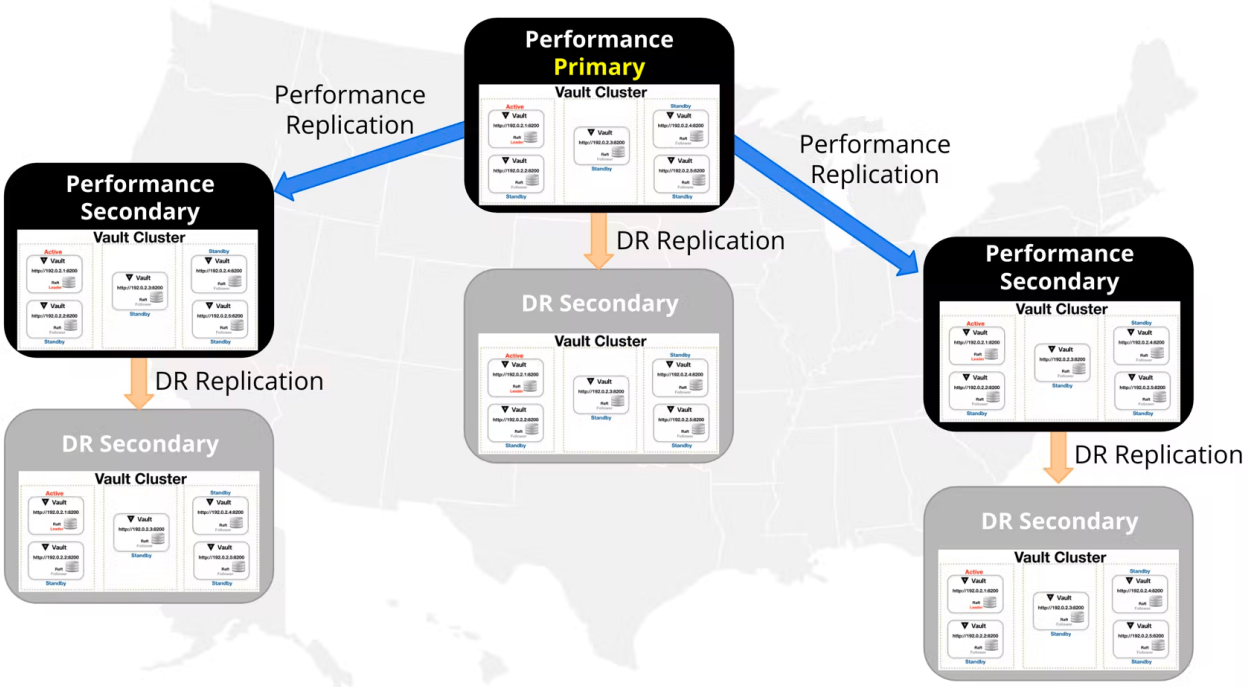


Figure 2: PR and DR Architecture

DR and PR replication function in a "one-to-many" architecture. A given DR or PR primary cluster can be associated with multiple DR or PR secondaries. Vault does not place a technical limit on the number of replication relationships that can be established with a given DR or PR primary cluster. However, each replica cluster does add overhead to the active node on the primary cluster, as many transactions must be duplicated to all secondary clusters. The number of PR secondaries that can be paired with a primary cluster without major performance degradation on the primary will ultimately be a function of the specific workload in that Vault environment. We have observed Vault enterprise customers deploying "10s" of PR replicas in production environments successfully.

All communication between primaries and secondaries is end-to-end encrypted with mutually-authenticated TLS sessions, setup via replication tokens which are exchanged during the configuration of the replication relationship.

Each cluster in a DR or PR replication relationship should be configured with an independent auto-unseal mechanism. This means that each secondary cluster does not need to use the same cloud key management service (KMS) unseal key or hardware security module (HSM) unseal key as the primary. Replica clusters that have been deployed across regions can make use of a KMS/HSM

unseal key in their local region/datacenter.

When configuring DR or PR replication relationships using auto-unseal mechanisms, the secondary cluster will inherit the recovery keys and recovery configuration of the primary cluster. It is important to ensure that the recovery keys from the primary cluster continue to be kept in a secure manner.

Interaction with Vault clients

Vault Enterprise PR secondary clusters keep track of their own tokens and leases but share the underlying configuration, policies, and supporting secrets (K/V values, encryption keys for transit, etc). If a user action would modify the underlying shared state, the secondary forwards the request to the primary to be handled; this forwarding is fast and is transparent to the client. In most common use cases, the majority of client requests to Vault clusters tend to be read-oriented or localized write requests that can be satisfied by a PR secondary without consulting the primary cluster. Examples of these types of requests include client authentications, reads to the K/V secret backend, encryption/decryption operations in the transit backend, issuance of PKI certificates (depending on engine configuration), and the creation of dynamic secret leases. Because the most common client operations can be handled locally by a PR cluster, this design allows Vault to scale horizontally with the number of secondaries.

Generally, when using Vault performance replication to scale a Vault deployment across multiple regions or data centers, Vault clients should be configured to communicate with their local performance replica cluster, and communications with more than one performance replica cluster will not be needed. In a rare use case where a Vault client needs to communicate with more than one performance replica cluster, you will need to consider the information in the following paragraphs outlining the behavior of Vault tokens when being used across multiple clusters.

Since each PR cluster keeps track of its own tokens and leases, Vault service tokens are not portable between replicated clusters. If a Vault client using service tokens needs to interact with more than one PR cluster, the client will not be able to reuse the same service token on multiple clusters. The client will need to re-authenticate and obtain a new token which is local to the specific PR cluster that it is interacting with.

If a client is using Vault batch tokens, the behavior can be different. Orphan batch tokens contain all the information needed to authenticate to any PR cluster in the replication relationship. Therefore, the same orphan batch token can be reused to authenticate across multiple PR clusters.

Interaction with disaster replication

When planning for a disaster scenario, it is important to understand how the two types of replication modes interact, and how performance replica clusters will behave in a DR promotion scenario.

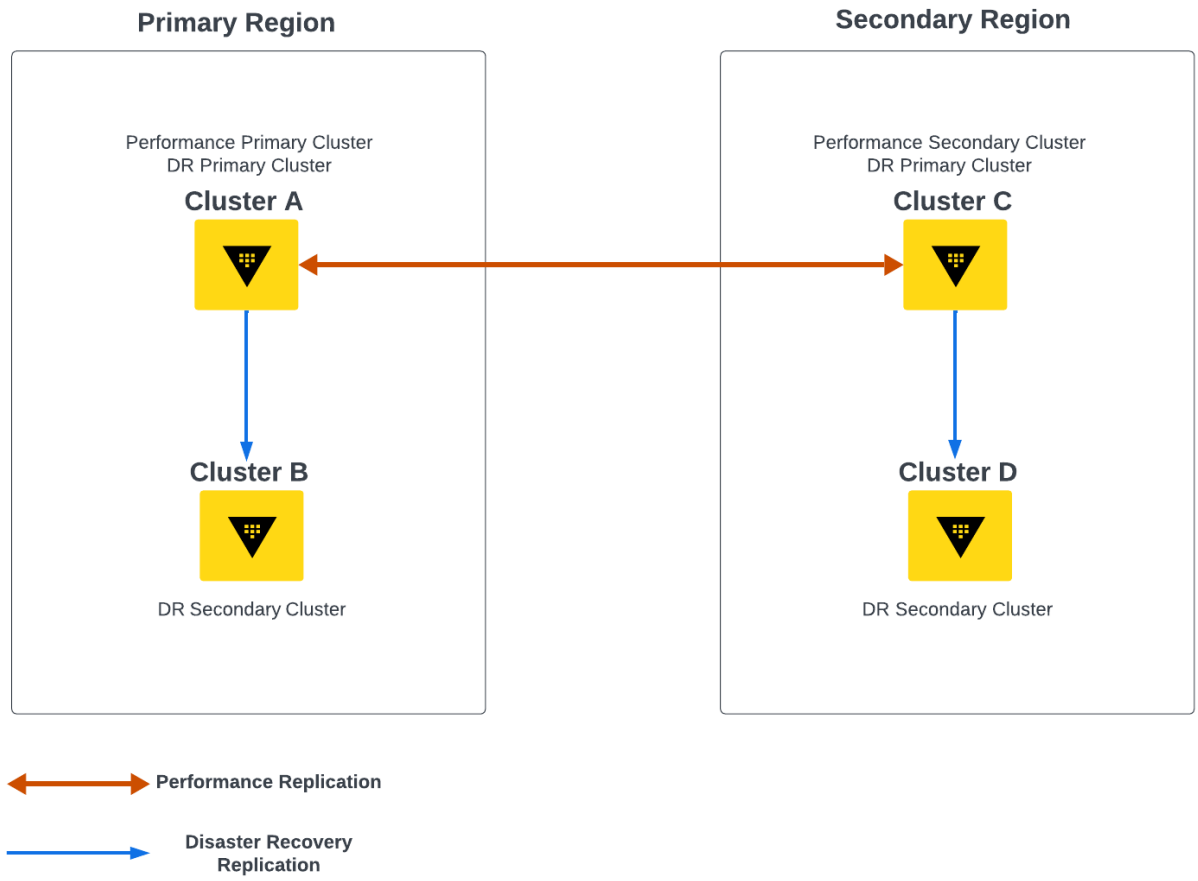


Figure 1. PR and DR Interaction

In the above diagram (Figure 1), Vault Enterprise is deployed across 2 data centers, each containing a PR primary or secondary cluster and each paired with a DR cluster. Consider the case where cluster A fails: in this scenario the Vault operators will promote cluster B to become a DR and PR primary cluster.

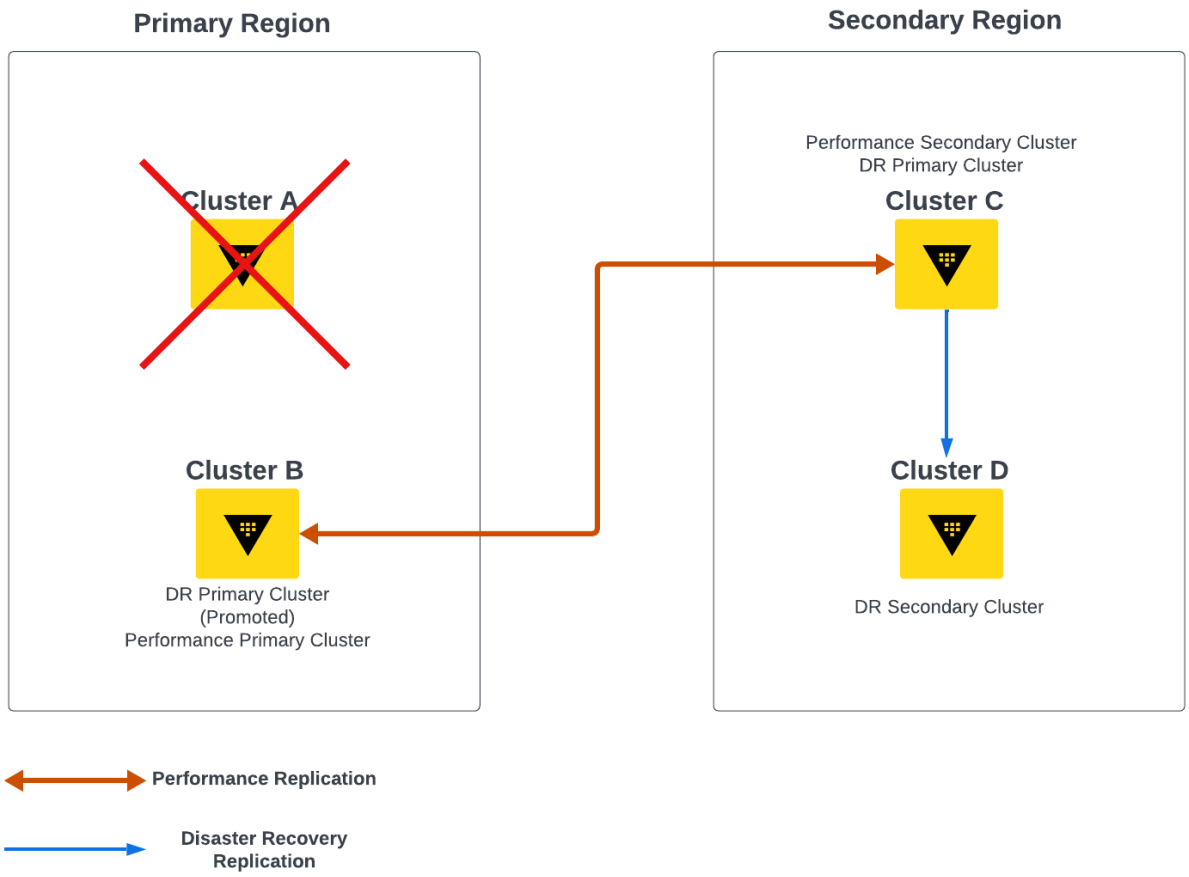


Figure 2. DR Scenario

When cluster B is promoted to be a DR Primary, performance replication needs to be reestablished between cluster B and cluster C. Cluster C accomplishes this by keeping a list of DR secondary nodes associated with cluster A. These addresses are stored in an internal reference known as failover_addr on cluster C when replication is established. During normal operations, clusters A and C will communicate without issues. When cluster A fails, cluster C will attempt to establish communication for a few moments, and then will begin attempting to connect with the addresses stored in the failover_addr reference. Once cluster B is promoted to DR Primary, cluster C will be able to reestablish communication.

In most scenarios, performance replication should be able to reestablish automatically with a new DR primary cluster based on the procedure above. However, this requires that the PR secondary nodes have network access to the promoted DR primary nodes over the Vault cluster

port (8201). It is important to ensure that this network connectivity requirement is considered during the design of your PR cluster. In scenarios where manual intervention may be necessary to update the cluster address used for replication on the PR secondary cluster, the /sys/replication/performance/secondary/update-primary API endpoint can be used to update the primary cluster address.

8.3 Setting up performance replication

> Note
>
> On HCP Vault dedicated, performance replica clusters are deployed through the HCP control plane. You will not need to configure the underlying infrastructure or go through the replication configuration if you are using HCP Vault Dedicated. Other considerations including those outlined above regarding interaction with Vault clients will still apply on HCP Vault Dedicated.

8.3.1 Prerequisites

- You have deployed a separate Vault cluster that is intended for performance replication
- The PR replica Vault cluster is initialized and unsealed
- You have a valid token for your primary cluster and PR replica cluster

Networking considerations

Performance replication makes use of two TCP ports for communication between clusters: the Vault API port (TCP-8200) and the cluster port (TCP-8201). When replication is being established, a two step process takes place:

- First, the leader node in the PR secondary cluster sends an authentication request over TCP-8200 to the PR primary cluster using the encoded leader API address and batch token contained in the encoded replication token.
- Next, the secondary obtains a list of cluster addresses from the primary leader node, and initiates replication traffic on the cluster port over TCP-8201 with the primary leader node.

Note

If the network path between the two clusters does not allow direct network traffic on the cluster port (TCP-8201) between all of the nodes in each cluster, then you will need to manually set the primary cluster address to override the default encoded cluster address value for replication when you enable replication on the primary. This is often required when the primary cluster is only accessible via a network path that includes a load balancer. Additionally, note that replication traffic on the Vault cluster port (TCP-8201) must be directed to the leader node of the primary cluster. If a load balancer is the only network path for replication traffic on the cluster port (TCP-8201), the load balancer should be configured via health checks through the cluster port (TCP-8200) to determine the current leader node.

TLS Considerations

In step two above, while establishing a replication connection over the cluster port, mutual TLS authentication takes place between the PR secondary and primary cluster leaders, and replication traffic commences via HTTP/2 gRPC. Therefore, no intermediate TLS termination (such as using an intermediate load balancer serving a TLS certificate) can be in place on port TCP-8201 between the primary and secondary cluster. Otherwise the mutual TLS authentication process will fail and replication will not be enabled.

8.3.2 Set up process

Refer to this tutorial for the steps to set up performance replication. At a high level, the steps are:

- Enable performance replication on the primary cluster, obtaining a PR secondary activation token
- Using the PR secondary activation token, enable performance replication on the secondary cluster
- Verify and monitor performance replication

Note

Vault does not replicate automated integrated storage snapshots as a part of performance replication. You must explicitly configure both the primary and secondary performance clusters to create individual automated snapshots. However, this is not the case for audit devices. Vault replicates the configuration of audit devices as part of performance replication. This means that the same directory (for file-based devices) or the same endpoint (for socket-based devices) must be reachable by all the nodes in the PR cluster.

9 Observability setup

Note

This page covers install-time observability wiring for Vault telemetry and health endpoints. For recurring monitoring, alert response, audit-log review, operational-log review, and incident response, see the Administration Guide: Monitoring and observability.

9.1 Telemetry endpoint setup

Enable Vault telemetry during deployment so platform monitoring systems can collect Vault metrics. Configure the `telemetry` stanza in the Vault server configuration, expose the Prometheus metrics endpoint only to approved monitoring systems, and confirm that the endpoint is reachable from the monitoring plane before handoff to day-2 operations.

9.2 Health endpoint setup

Configure load balancers, probes, and monitoring checks to use the Vault health API. The health endpoint reports the state of the Vault node being queried, so checks should account for active, standby, sealed, and performance standby states.

9.3 Handoff to administration

After telemetry and health endpoints are wired into the monitoring platform, use the Administration Guide to define dashboards, key metrics, alert thresholds, audit-log review, operational-log review,

and incident-response workflows.

10 Next steps

Follow HashiCorp guidance for Vault production hardening.

For further information on adopting Vault Enterprise within your organization according to HashiCorp Validated Designs, visit Vault: validated design guidance.

11 Changelog

11.1 2026-08

- Restructured the Vault Enterprise Solution Design Guide into the HVD 2.0 Vault Installation Guide as part of the Installation / Administration / User guide migration.

11.2 2026-03

- Add mlock configuration to config file examples

11.3 2026-01

- Fixed heading formatting in tabbed content for PDF generation
- Update meta descriptions
- Updated credits and contributors
- Remove AWS KMS seal configuration

11.4 2025-12

- Vault design updates

11.5 2025-10

- Update mlock guidance to match docs

11.6 2025-05

- Removed document structure section

11.7 2025-04

- Update init command to use correct pgp keys argument

11.8 2025-01

- Fixed systemd configuration file paths

11.9 2024-02

- GA release

12 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Adam Cavaliere
- Anna Chen
- Finn Dannin
- Hari Sankaran
- Jan Repnak
- John Duke

- Justin Clayton
- Kalen Arndt
- Mara Hammond
- Mark Lewis
- Nalini Dayma
- Neil Dahlke
- Nick Wong
- Prakash Manglanathan
- Rowan Smith
- Sean Doyle